

ABSTRACT FACTORY

Object Creational

Intent

Provide an interface for creating families of related or dependent objects without specifying their concrete classes.

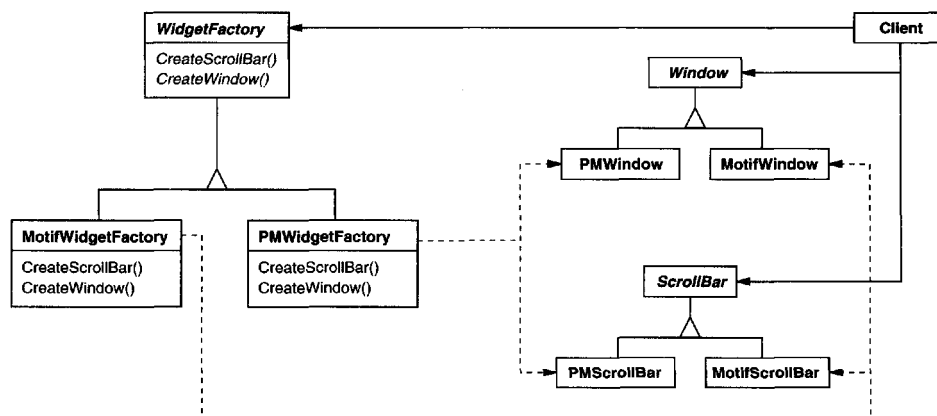
Also Known As

Kit

Motivation

Consider a user interface toolkit that supports multiple look-and-feel standards, such as Motif and Presentation Manager. Different look-and-feels define different appearances and behaviors for user interface “widgets” like scroll bars, windows, and buttons. To be portable across look-and-feel standards, an application should not hard-code its widgets for a particular look and feel. Instantiating look-and-feel-specific classes of widgets throughout the application makes it hard to change the look and feel later.

We can solve this problem by defining an abstract WidgetFactory class that declares an interface for creating each basic kind of widget. There’s also an abstract class for each kind of widget, and concrete subclasses implement widgets for specific look-and-feel standards. WidgetFactory’s interface has an operation that returns a new widget object for each abstract widget class. Clients call these operations to obtain widget instances, but clients aren’t aware of the concrete classes they’re using. Thus clients stay independent of the prevailing look and feel.



There is a concrete subclass of `WidgetFactory` for each look-and-feel standard. Each subclass implements the operations to create the appropriate widget for the look and feel. For example, the `CreateScrollBar` operation on the `MotifWidgetFactory` instantiates and returns a Motif scroll bar, while the corresponding operation on the `PMWidgetFactory` returns a scroll bar for Presentation Manager. Clients create widgets solely through the `WidgetFactory` interface and have no knowledge of the classes that implement widgets for a particular look and feel. In other words, clients only have to commit to an interface defined by an abstract class, not a particular concrete class.

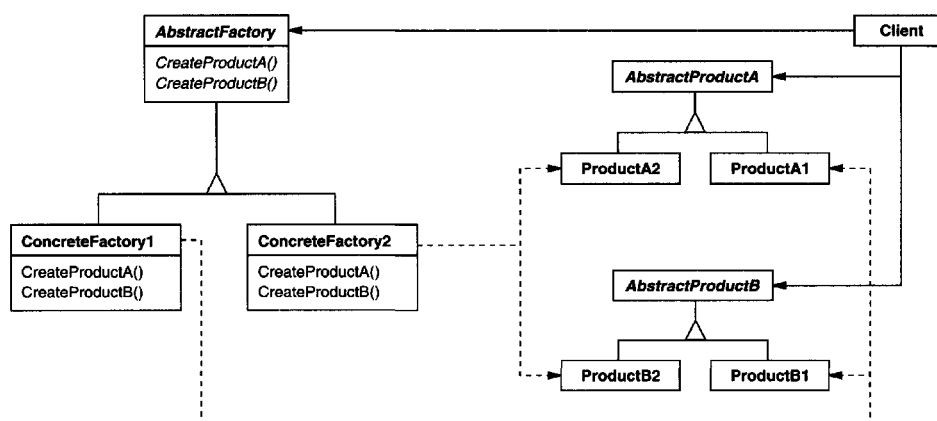
A `WidgetFactory` also enforces dependencies between the concrete widget classes. A Motif scroll bar should be used with a Motif button and a Motif text editor, and that constraint is enforced automatically as a consequence of using a `MotifWidgetFactory`.

Applicability

Use the Abstract Factory pattern when

- a system should be independent of how its products are created, composed, and represented.
- a system should be configured with one of multiple families of products.
- a family of related product objects is designed to be used together, and you need to enforce this constraint.
- you want to provide a class library of products, and you want to reveal just their interfaces, not their implementations.

Structure



Participants

- **AbstractFactory** (WidgetFactory)
 - declares an interface for operations that create abstract product objects.
- **ConcreteFactory** (MotifWidgetFactory, PMWidgetFactory)
 - implements the operations to create concrete product objects.
- **AbstractProduct** (Window, ScrollBar)
 - declares an interface for a type of product object.
- **ConcreteProduct** (MotifWindow, MotifScrollBar)
 - defines a product object to be created by the corresponding concrete factory.
 - implements the AbstractProduct interface.
- **Client**
 - uses only interfaces declared by AbstractFactory and AbstractProduct classes.

Collaborations

- Normally a single instance of a ConcreteFactory class is created at run-time. This concrete factory creates product objects having a particular implementation. To create different product objects, clients should use a different concrete factory.
- AbstractFactory defers creation of product objects to its ConcreteFactory subclass.

Consequences

The Abstract Factory pattern has the following benefits and liabilities:

1. *It isolates concrete classes.* The Abstract Factory pattern helps you control the classes of objects that an application creates. Because a factory encapsulates the responsibility and the process of creating product objects, it isolates clients from implementation classes. Clients manipulate instances through their abstract interfaces. Product class names are isolated in the implementation of the concrete factory; they do not appear in client code.
2. *It makes exchanging product families easy.* The class of a concrete factory appears only once in an application—that is, where it's instantiated. This makes it easy to change the concrete factory an application uses. It can use different product configurations simply by changing the concrete factory. Because an abstract factory creates a complete family of products, the whole product family changes at once. In our user interface example, we can switch from Motif widgets to Presentation Manager widgets simply by switching the corresponding factory objects and recreating the interface.

3. *It promotes consistency among products.* When product objects in a family are designed to work together, it's important that an application use objects from only one family at a time. AbstractFactory makes this easy to enforce.
4. *Supporting new kinds of products is difficult.* Extending abstract factories to produce new kinds of Products isn't easy. That's because the AbstractFactory interface fixes the set of products that can be created. Supporting new kinds of products requires extending the factory interface, which involves changing the AbstractFactory class and all of its subclasses. We discuss one solution to this problem in the Implementation section.

Implementation

Here are some useful techniques for implementing the Abstract Factory pattern.

1. *Factories as singletons.* An application typically needs only one instance of a ConcreteFactory per product family. So it's usually best implemented as a Singleton (127).
2. *Creating the products.* AbstractFactory only declares an *interface* for creating products. It's up to ConcreteProduct subclasses to actually create them. The most common way to do this is to define a factory method (see Factory Method (107)) for each product. A concrete factory will specify its products by overriding the factory method for each. While this implementation is simple, it requires a new concrete factory subclass for each product family, even if the product families differ only slightly.

If many product families are possible, the concrete factory can be implemented using the Prototype (117) pattern. The concrete factory is initialized with a prototypical instance of each product in the family, and it creates a new product by cloning its prototype. The Prototype-based approach eliminates the need for a new concrete factory class for each new product family.

Here's a way to implement a Prototype-based factory in Smalltalk. The concrete factory stores the prototypes to be cloned in a dictionary called `partCatalog`. The method `make:` retrieves the prototype and clones it:

```
make: partName
    ^ (partCatalog at: partName) copy
```

The concrete factory has a method for adding parts to the catalog.

```
addPart: partTemplate named: partName
    partCatalog at: partName put: partTemplate
```

Prototypes are added to the factory by identifying them with a symbol:

```
aFactory addPart: aPrototype named: #ACMEWidget
```

A variation on the Prototype-based approach is possible in languages that treat classes as first-class objects (Smalltalk and Objective C, for example). You

can think of a class in these languages as a degenerate factory that creates only one kind of product. You can store *classes* inside a concrete factory that create the various concrete products in variables, much like prototypes. These classes create new instances on behalf of the concrete factory. You define a new factory by initializing an instance of a concrete factory with *classes* of products rather than by subclassing. This approach takes advantage of language characteristics, whereas the pure Prototype-based approach is language-independent.

Like the Prototype-based factory in Smalltalk just discussed, the class-based version will have a single instance variable `partCatalog`, which is a dictionary whose key is the name of the part. Instead of storing prototypes to be cloned, `partCatalog` stores the classes of the products. The method `make:` now looks like this:

```
make: partName
    ^ (partCatalog at: partName) new
```

3. *Defining extensible factories.* AbstractFactory usually defines a different operation for each kind of product it can produce. The kinds of products are encoded in the operation signatures. Adding a new kind of product requires changing the AbstractFactory interface and all the classes that depend on it.

A more flexible but less safe design is to add a parameter to operations that create objects. This parameter specifies the kind of object to be created. It could be a class identifier, an integer, a string, or anything else that identifies the kind of product. In fact with this approach, AbstractFactory only needs a single “Make” operation with a parameter indicating the kind of object to create. This is the technique used in the Prototype- and the class-based abstract factories discussed earlier.

This variation is easier to use in a dynamically typed language like Smalltalk than in a statically typed language like C++. You can use it in C++ only when all objects have the same abstract base class or when the product objects can be safely coerced to the correct type by the client that requested them. The implementation section of Factory Method (107) shows how to implement such parameterized operations in C++.

But even when no coercion is needed, an inherent problem remains: All products are returned to the client with the *same* abstract interface as given by the return type. The client will not be able to differentiate or make safe assumptions about the class of a product. If clients need to perform subclass-specific operations, they won’t be accessible through the abstract interface. Although the client could perform a downcast (e.g., with `dynamic_cast` in C++), that’s not always feasible or safe, because the downcast can fail. This is the classic trade-off for a highly flexible and extensible interface.

Sample Code

We'll apply the Abstract Factory pattern to creating the mazes we discussed at the beginning of this chapter.

Class `MazeFactory` can create components of mazes. It builds rooms, walls, and doors between rooms. It might be used by a program that reads plans for mazes from a file and builds the corresponding maze. Or it might be used by a program that builds mazes randomly. Programs that build mazes take a `MazeFactory` as an argument so that the programmer can specify the classes of rooms, walls, and doors to construct.

```
class MazeFactory {
public:
    MazeFactory();

    virtual Maze* MakeMaze() const
    { return new Maze; }
    virtual Wall* MakeWall() const
    { return new Wall; }
    virtual Room* MakeRoom(int n) const
    { return new Room(n); }
    virtual Door* MakeDoor(Room* r1, Room* r2) const
    { return new Door(r1, r2); }
};
```

Recall that the member function `CreateMaze` (page 84) builds a small maze consisting of two rooms with a door between them. `CreateMaze` hard-codes the class names, making it difficult to create mazes with different components.

Here's a version of `CreateMaze` that remedies that shortcoming by taking a `MazeFactory` as a parameter:

```
Maze* MazeGame::CreateMaze (MazeFactory& factory) {
    Maze* aMaze = factory.MakeMaze();
    Room* r1 = factory.MakeRoom(1);
    Room* r2 = factory.MakeRoom(2);
    Door* aDoor = factory.MakeDoor(r1, r2);

    aMaze->AddRoom(r1);
    aMaze->AddRoom(r2);

    r1->SetSide(North, factory.MakeWall());
    r1->SetSide(East, aDoor);
    r1->SetSide(South, factory.MakeWall());
    r1->SetSide(West, factory.MakeWall());
```

```

        r2->SetSide(North, factory.MakeWall());
        r2->SetSide(East, factory.MakeWall());
        r2->SetSide(South, factory.MakeWall());
        r2->SetSide(West, aDoor);

        return aMaze;
    }

```

We can create `EnchantedMazeFactory`, a factory for enchanted mazes, by subclassing `MazeFactory`. `EnchantedMazeFactory` will override different member functions and return different subclasses of `Room`, `Wall`, etc.

```

class EnchantedMazeFactory : public MazeFactory {
public:
    EnchantedMazeFactory();

    virtual Room* MakeRoom(int n) const
        { return new EnchantedRoom(n, CastSpell()); }

    virtual Door* MakeDoor(Room* r1, Room* r2) const
        { return new DoorNeedingSpell(r1, r2); }

protected:
    Spell* CastSpell() const;
};

```

Now suppose we want to make a maze game in which a room can have a bomb set in it. If the bomb goes off, it will damage the walls (at least). We can make a subclass of `Room` keep track of whether the room has a bomb in it and whether the bomb has gone off. We'll also need a subclass of `Wall` to keep track of the damage done to the wall. We'll call these classes `RoomWithABomb` and `BombedWall`.

The last class we'll define is `BombedMazeFactory`, a subclass of `MazeFactory` that ensures walls are of class `BombedWall` and rooms are of class `RoomWithABomb`. `BombedMazeFactory` only needs to override two functions:

```

Wall* BombedMazeFactory::MakeWall () const {
    return new BombedWall;
}

Room* BombedMazeFactory::MakeRoom(int n) const {
    return new RoomWithABomb(n);
}

```

To build a simple maze that can contain bombs, we simply call `CreateMaze` with a `BombedMazeFactory`.

```

MazeGame game;
BombedMazeFactory factory;

game.CreateMaze(factory);

```

CreateMaze can take an instance of EnchantedMazeFactory just as well to build enchanted mazes.

Notice that the MazeFactory is just a collection of factory methods. This is the most common way to implement the Abstract Factory pattern. Also note that MazeFactory is not an abstract class; thus it acts as both the AbstractFactory *and* the ConcreteFactory. This is another common implementation for simple applications of the Abstract Factory pattern. Because the MazeFactory is a concrete class consisting entirely of factory methods, it's easy to make a new MazeFactory by making a subclass and overriding the operations that need to change.

CreateMaze used the SetSide operation on rooms to specify their sides. If it creates rooms with a BombedMazeFactory, then the maze will be made up of RoomWithABomb objects with BombedWall sides. If RoomWithABomb had to access a subclass-specific member of BombedWall, then it would have to cast a reference to its walls from Wall* to BombedWall*. This downcasting is safe as long as the argument *is* in fact a BombedWall, which is guaranteed to be true if walls are built solely with a BombedMazeFactory.

Dynamically typed languages such as Smalltalk don't require downcasting, of course, but they might produce run-time errors if they encounter a Wall where they expect a *subclass* of Wall. Using Abstract Factory to build walls helps prevent these run-time errors by ensuring that only certain kinds of walls can be created.

Let's consider a Smalltalk version of MazeFactory, one with a single make operation that takes the kind of object to make as a parameter. Moreover, the concrete factory stores the classes of the products it creates.

First, we'll write an equivalent of CreateMaze in Smalltalk:

```
createMaze: aFactory
| room1 room2 aDoor |
room1 := (aFactory make: #room) number: 1.
room2 := (aFactory make: #room) number: 2.
aDoor := (aFactory make: #door) from: room1 to: room2.
room1 atSide: #north put: (aFactory make: #wall).
room1 atSide: #east put: aDoor.
room1 atSide: #south put: (aFactory make: #wall).
room1 atSide: #west put: (aFactory make: #wall).
room2 atSide: #north put: (aFactory make: #wall).
room2 atSide: #east put: (aFactory make: #wall).
room2 atSide: #south put: (aFactory make: #wall).
room2 atSide: #west put: aDoor.
^ Maze new addRoom: room1; addRoom: room2; yourself
```

As we discussed in the Implementation section, MazeFactory needs only a single instance variable partCatalog to provide a dictionary whose key is the class of the component. Also recall how we implemented the make: method:

```
make: partName
^ (partCatalog at: partName) new
```

Now we can create a `MazeFactory` and use it to implement `createMaze`. We'll create the factory using a method `createMazeFactory` of class `MazeGame`.

```
createMazeFactory
^ (MazeFactory new
  addPart: Wall named: #wall;
  addPart: Room named: #room;
  addPart: Door named: #door;
  yourself)
```

A `BombedMazeFactory` or `EnchantedMazeFactory` is created by associating different classes with the keys. For example, an `EnchantedMazeFactory` could be created like this:

```
createMazeFactory
^ (MazeFactory new
  addPart: Wall named: #wall;
  addPart: EnchantedRoom named: #room;
  addPart: DoorNeedingSpell named: #door;
  yourself)
```

Known Uses

InterViews uses the “Kit” suffix [Lin92] to denote `AbstractFactory` classes. It defines `WidgetKit` and `DialogKit` abstract factories for generating look-and-feel-specific user interface objects. *InterViews* also includes a `LayoutKit` that generates different composition objects depending on the layout desired. For example, a layout that is conceptually horizontal may require different composition objects depending on the document's orientation (portrait or landscape).

ET++ [WGM88] uses the Abstract Factory pattern to achieve portability across different window systems (X Windows and *SunView*, for example). The `WindowSystem` abstract base class defines the interface for creating objects that represent window system resources (`MakeWindow`, `MakeFont`, `MakeColor`, for example). Concrete subclasses implement the interfaces for a specific window system. At run-time, *ET++* creates an instance of a concrete `WindowSystem` subclass that creates concrete system resource objects.

Related Patterns

`AbstractFactory` classes are often implemented with factory methods (Factory Method (107)), but they can also be implemented using Prototype (117).

A concrete factory is often a singleton (Singleton (127)).

Intent

Decouple an abstraction from its implementation so that the two can vary independently.

Also Known As

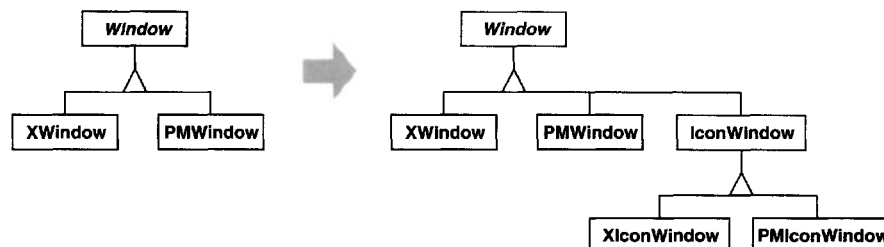
Handle/Body

Motivation

When an abstraction can have one of several possible implementations, the usual way to accommodate them is to use inheritance. An abstract class defines the interface to the abstraction, and concrete subclasses implement it in different ways. But this approach isn't always flexible enough. Inheritance binds an implementation to the abstraction permanently, which makes it difficult to modify, extend, and reuse abstractions and implementations independently.

Consider the implementation of a portable Window abstraction in a user interface toolkit. This abstraction should enable us to write applications that work on both the X Window System and IBM's Presentation Manager (PM), for example. Using inheritance, we could define an abstract class Window and subclasses XWindow and PMWindow that implement the Window interface for the different platforms. But this approach has two drawbacks:

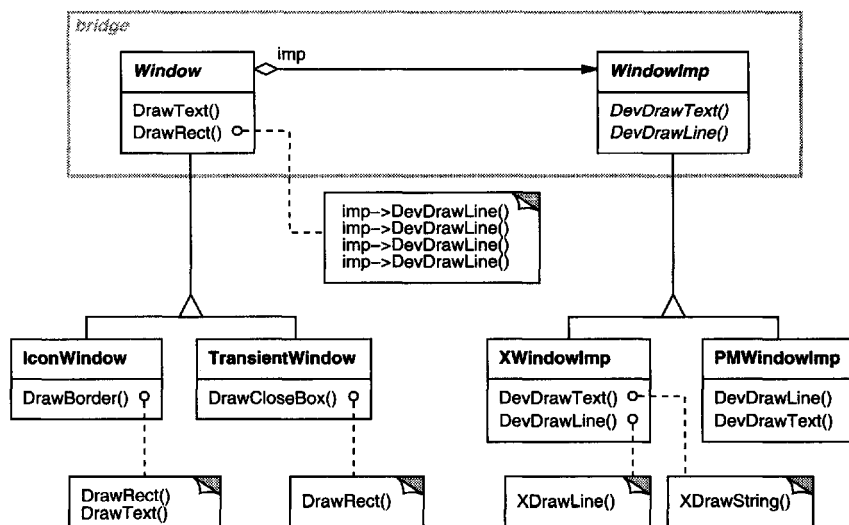
1. It's inconvenient to extend the Window abstraction to cover different kinds of windows or new platforms. Imagine an IconWindow subclass of Window that specializes the Window abstraction for icons. To support IconWindows for both platforms, we have to implement *two* new classes, XIconWindow and PMIconWindow. Worse, we'll have to define two classes for *every* kind of window. Supporting a third platform requires yet another new Window subclass for every kind of window.



2. It makes client code platform-dependent. Whenever a client creates a window, it instantiates a concrete class that has a specific implementation. For example, creating an XWindow object binds the Window abstraction to the X Window implementation, which makes the client code dependent on the X Window implementation. This, in turn, makes it harder to port the client code to other platforms.

Clients should be able to create a window without committing to a concrete implementation. Only the window implementation should depend on the platform on which the application runs. Therefore client code should instantiate windows without mentioning specific platforms.

The Bridge pattern addresses these problems by putting the Window abstraction and its implementation in separate class hierarchies. There is one class hierarchy for window interfaces (Window, IconWindow, TransientWindow) and a separate hierarchy for platform-specific window implementations, with WindowImp as its root. The XWindowImp subclass, for example, provides an implementation based on the X Window System.



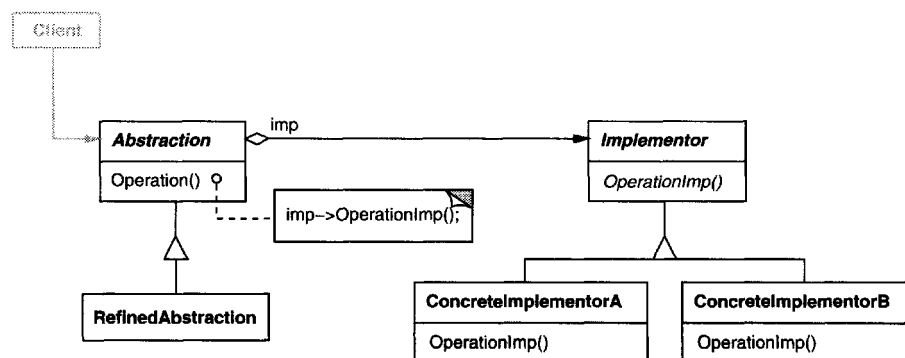
All operations on Window subclasses are implemented in terms of abstract operations from the WindowImp interface. This decouples the window abstractions from the various platform-specific implementations. We refer to the relationship between Window and WindowImp as a **bridge**, because it bridges the abstraction and its implementation, letting them vary independently.

Applicability

Use the Bridge pattern when

- you want to avoid a permanent binding between an abstraction and its implementation. This might be the case, for example, when the implementation must be selected or switched at run-time.
- both the abstractions and their implementations should be extensible by subclassing. In this case, the Bridge pattern lets you combine the different abstractions and implementations and extend them independently.
- changes in the implementation of an abstraction should have no impact on clients; that is, their code should not have to be recompiled.
- (C++) you want to hide the implementation of an abstraction completely from clients. In C++ the representation of a class is visible in the class interface.
- you have a proliferation of classes as shown earlier in the first Motivation diagram. Such a class hierarchy indicates the need for splitting an object into two parts. Rumbaugh uses the term “nested generalizations” [RBP⁺91] to refer to such class hierarchies.
- you want to share an implementation among multiple objects (perhaps using reference counting), and this fact should be hidden from the client. A simple example is Coplien’s String class [Cop92], in which multiple objects can share the same string representation (StringRep).

Structure



Participants

- **Abstraction** (Window)
 - defines the abstraction's interface.
 - maintains a reference to an object of type Implementor.
- **RefinedAbstraction** (IconWindow)
 - Extends the interface defined by Abstraction.
- **Implementor** (WindowImp)
 - defines the interface for implementation classes. This interface doesn't have to correspond exactly to Abstraction's interface; in fact the two interfaces can be quite different. Typically the Implementor interface provides only primitive operations, and Abstraction defines higher-level operations based on these primitives.
- **ConcreteImplementor** (XWindowImp, PMWindowImp)
 - implements the Implementor interface and defines its concrete implementation.

Collaborations

- Abstraction forwards client requests to its Implementor object.

Consequences

The Bridge pattern has the following consequences:

1. *Decoupling interface and implementation.* An implementation is not bound permanently to an interface. The implementation of an abstraction can be configured at run-time. It's even possible for an object to change its implementation at run-time.
Decoupling Abstraction and Implementor also eliminates compile-time dependencies on the implementation. Changing an implementation class doesn't require recompiling the Abstraction class and its clients. This property is essential when you must ensure binary compatibility between different versions of a class library.
Furthermore, this decoupling encourages layering that can lead to a better-structured system. The high-level part of a system only has to know about Abstraction and Implementor.
2. *Improved extensibility.* You can extend the Abstraction and Implementor hierarchies independently.
3. *Hiding implementation details from clients.* You can shield clients from implementation details, like the sharing of implementor objects and the accompanying reference count mechanism (if any).

Implementation

Consider the following implementation issues when applying the Bridge pattern:

1. *Only one Implementor.* In situations where there's only one implementation, creating an abstract Implementor class isn't necessary. This is a degenerate case of the Bridge pattern; there's a one-to-one relationship between Abstraction and Implementor. Nevertheless, this separation is still useful when a change in the implementation of a class must not affect its existing clients—that is, they shouldn't have to be recompiled, just relinked.

Carolan [Car89] uses the term “Cheshire Cat” to describe this separation. In C++, the class interface of the Implementor class can be defined in a private header file that isn't provided to clients. This lets you hide an implementation of a class completely from its clients.

2. *Creating the right Implementor object.* How, when, and where do you decide which Implementor class to instantiate when there's more than one?

If Abstraction knows about all ConcreteImplementor classes, then it can instantiate one of them in its constructor; it can decide between them based on parameters passed to its constructor. If, for example, a collection class supports multiple implementations, the decision can be based on the size of the collection. A linked list implementation can be used for small collections and a hash table for larger ones.

Another approach is to choose a default implementation initially and change it later according to usage. For example, if the collection grows bigger than a certain threshold, then it switches its implementation to one that's more appropriate for a large number of items.

It's also possible to delegate the decision to another object altogether. In the Window/WindowImp example, we can introduce a factory object (see Abstract Factory (87)) whose sole duty is to encapsulate platform-specifics. The factory knows what kind of WindowImp object to create for the platform in use; a Window simply asks it for a WindowImp, and it returns the right kind. A benefit of this approach is that Abstraction is not coupled directly to any of the Implementor classes.

3. *Sharing implementors.* Coplien illustrates how the Handle/Body idiom in C++ can be used to share implementations among several objects [Cop92]. The Body stores a reference count that the Handle class increments and decrements. The code for assigning handles with shared bodies has the following general form:

```

Handle& Handle::operator= (const Handle& other) {
    other._body->Ref();
    _body->Unref();

    if (_body->RefCount() == 0) {
        delete _body;
    }
    _body = other._body;

    return *this;
}

```

4. *Using multiple inheritance.* You can use multiple inheritance in C++ to combine an interface with its implementation [Mar91]. For example, a class can inherit publicly from Abstraction and privately from a ConcreteImplementor. But because this approach relies on static inheritance, it binds an implementation permanently to its interface. Therefore you can't implement a true Bridge with multiple inheritance—at least not in C++.

Sample Code

The following C++ code implements the Window/WindowImp example from the Motivation section. The Window class defines the window abstraction for client applications:

```

class Window {
public:
    Window(View* contents);

    // requests handled by window
    virtual void DrawContents();

    virtual void Open();
    virtual void Close();
    virtual void Iconify();
    virtual void Deiconify();

    // requests forwarded to implementation
    virtual void SetOrigin(const Point& at);
    virtual void SetExtent(const Point& extent);
    virtual void Raise();
    virtual void Lower();

    virtual void DrawLine(const Point&, const Point&);
    virtual void DrawRect(const Point&, const Point&);
    virtual void DrawPolygon(const Point[], int n);
    virtual void DrawText(const char*, const Point&);

protected:
    WindowImp* GetWindowImp();
    View* GetView();
}

```

```
private:
    WindowImp* _imp;
    View* _contents; // the window's contents
};
```

Window maintains a reference to a WindowImp, the abstract class that declares an interface to the underlying windowing system.

```
class WindowImp {
public:
    virtual void ImpTop() = 0;
    virtual void ImpBottom() = 0;
    virtual void ImpSetExtent(const Point&) = 0;
    virtual void ImpSetOrigin(const Point&) = 0;

    virtual void DeviceRect(Coord, Coord, Coord, Coord) = 0;
    virtual void DeviceText(const char*, Coord, Coord) = 0;
    virtual void DeviceBitmap(const char*, Coord, Coord) = 0;
    // lots more functions for drawing on windows...
protected:
    WindowImp();
};
```

Subclasses of Window define the different kinds of windows the application might use, such as application windows, icons, transient windows for dialogs, floating palettes of tools, and so on.

For example, ApplicationWindow will implement DrawContents to draw the View instance it stores:

```
class ApplicationWindow : public Window {
public:
    // ...
    virtual void DrawContents();
};

void ApplicationWindow::DrawContents () {
    GetView()->DrawOn(this);
}
```

IconWindow stores the name of a bitmap for the icon it displays...

```
class IconWindow : public Window {
public:
    // ...
    virtual void DrawContents();
private:
    const char* _bitmapName;
};
```

...and it implements DrawContents to draw the bitmap on the window:

```

void IconWindow::DrawContents() {
    WindowImp* imp = GetWindowImp();
    if (imp != 0) {
        imp->DeviceBitmap(_bitmapName, 0.0, 0.0);
    }
}

```

Many other variations of Window are possible. A *TransientWindow* may need to communicate with the window that created it during the dialog; hence it keeps a reference to that window. A *PaletteWindow* always floats above other windows. An *IconDockWindow* holds *IconWindows* and arranges them neatly.

Window operations are defined in terms of the *WindowImp* interface. For example, *DrawRect* extracts four coordinates from its two *Point* parameters before calling the *WindowImp* operation that draws the rectangle in the window:

```

void Window::DrawRect (const Point& p1, const Point& p2) {
    WindowImp* imp = GetWindowImp();
    imp->DeviceRect(p1.X(), p1.Y(), p2.X(), p2.Y());
}

```

Concrete subclasses of *WindowImp* support different window systems. The *XWindowImp* subclass supports the X Window System:

```

class XWindowImp : public WindowImp {
public:
    XWindowImp();

    virtual void DeviceRect(Coord, Coord, Coord, Coord);
    // remainder of public interface...

private:
    // lots of X window system-specific state, including:
    Display* _dpy;
    Drawable _winid; // window id
    GC _gc;          // window graphic context
};

```

For Presentation Manager (PM), we define a *PMWindowImp* class:

```

class PMWindowImp : public WindowImp {
public:
    PMWindowImp();
    virtual void DeviceRect(Coord, Coord, Coord, Coord);

    // remainder of public interface...

private:
    // lots of PM window system-specific state, including:
    HPS _hps;
};

```

These subclasses implement *WindowImp* operations in terms of window system primitives. For example, *DeviceRect* is implemented for X as follows:

```

void XWindowImp::DeviceRect (
    Coord x0, Coord y0, Coord x1, Coord y1
) {
    int x = round(min(x0, x1));
    int y = round(min(y0, y1));
    int w = round(abs(x0 - x1));
    int h = round(abs(y0 - y1));
    XDrawRectangle(_dpy, _winid, _gc, x, y, w, h);
}

```

The PM implementation might look like this:

```

void PMWindowImp::DeviceRect (
    Coord x0, Coord y0, Coord x1, Coord y1
) {
    Coord left = min(x0, x1);
    Coord right = max(x0, x1);
    Coord bottom = min(y0, y1);
    Coord top = max(y0, y1);

    PPOINTL point[4];

    point[0].x = left;    point[0].y = top;
    point[1].x = right;   point[1].y = top;
    point[2].x = right;   point[2].y = bottom;
    point[3].x = left;    point[3].y = bottom;

    if (
        (GpiBeginPath(_hps, 1L) == false) ||
        (GpiSetCurrentPosition(_hps, &point[3]) == false) ||
        (GpiPolyLine(_hps, 4L, point) == GPI_ERROR) ||
        (GpiEndPath(_hps) == false)
    ) {
        // report error

    } else {
        GpiStrokePath(_hps, 1L, 0L);
    }
}

```

How does a window obtain an instance of the right WindowImp subclass? We'll assume Window has that responsibility in this example. Its GetWindowImp operation gets the right instance from an abstract factory (see Abstract Factory (87)) that effectively encapsulates all window system specifics.

```

WindowImp* Window::GetWindowImp () {
    if (_imp == 0) {
        _imp = WindowSystemFactory::Instance()->MakeWindowImp();
    }
    return _imp;
}

```

`WindowSystemFactory::Instance()` returns an abstract factory that manufactures all window system-specific objects. For simplicity, we've made it a Singleton (127) and have let the `Window` class access the factory directly.

Known Uses

The `Window` example above comes from ET++ [WGM88]. In ET++, `WindowImp` is called "WindowPort" and has subclasses such as `XWindowPort` and `SunWindowPort`. The `Window` object creates its corresponding Implementor object by requesting it from an abstract factory called "WindowSystem." `WindowSystem` provides an interface for creating platform-specific objects such as fonts, cursors, bitmaps, and so forth.

The ET++ `Window/WindowPort` design extends the Bridge pattern in that the `WindowPort` also keeps a reference back to the `Window`. The `WindowPort` implementor class uses this reference to notify `Window` about `WindowPort`-specific events: the arrival of input events, window resizes, etc.

Both Coplien [Cop92] and Stroustrup [Str91] mention `Handle` classes and give some examples. Their examples emphasize memory management issues like sharing string representations and support for variable-sized objects. Our focus is more on supporting independent extension of both an abstraction and its implementation.

`libg++` [Lea88] defines classes that implement common data structures, such as `Set`, `LinkedSet`, `HashSet`, `LinkedList`, and `HashTable`. `Set` is an abstract class that defines a set abstraction, while `LinkedList` and `HashTable` are concrete implementors for a linked list and a hash table, respectively. `LinkedSet` and `HashSet` are `Set` implementors that bridge between `Set` and their concrete counterparts `LinkedList` and `HashTable`. This is an example of a degenerate bridge, because there's no abstract Implementor class.

NeXT's `AppKit` [Add94] uses the Bridge pattern in the implementation and display of graphical images. An image can be represented in several different ways. The optimal display of an image depends on the properties of a display device, specifically its color capabilities and its resolution. Without help from `AppKit`, developers would have to determine which implementation to use under various circumstances in every application.

To relieve developers of this responsibility, `AppKit` provides an `NXImage/NXImageRep` bridge. `NXImage` defines the interface for handling images. The implementation of images is defined in a separate `NXImageRep` class hierarchy having subclasses such as `NXEPSImageRep`, `NXCachedImageRep`, and `NXBitmapImageRep`. `NXImage` maintains a reference to one or more `NXImageRep` objects. If there is more than one image implementation, then `NXImage` selects the most appropriate one for the current display device. `NXImage` is even capable of converting one implementation to another if necessary. The interesting

aspect of this Bridge variant is that `NXImage` can store more than one `NXImageRep` implementation at a time.

Related Patterns

An Abstract Factory (87) can create and configure a particular Bridge.

The Adapter (139) pattern is geared toward making unrelated classes work together. It is usually applied to systems after they're designed. Bridge, on the other hand, is used up-front in a design to let abstractions and implementations vary independently.

COMMAND

Object Behavioral

Intent

Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

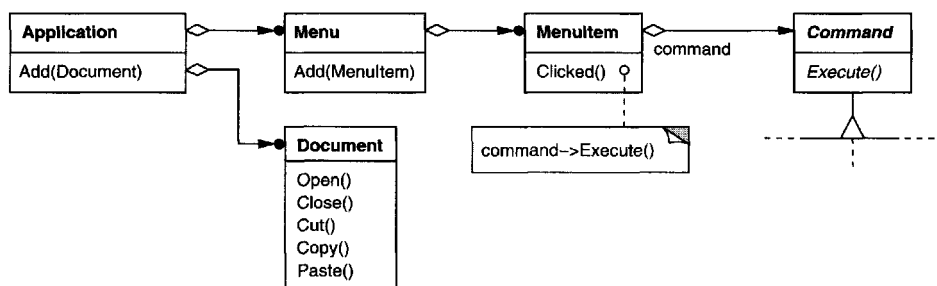
Also Known As

Action, Transaction

Motivation

Sometimes it's necessary to issue requests to objects without knowing anything about the operation being requested or the receiver of the request. For example, user interface toolkits include objects like buttons and menus that carry out a request in response to user input. But the toolkit can't implement the request explicitly in the button or menu, because only applications that use the toolkit know what should be done on which object. As toolkit designers we have no way of knowing the receiver of the request or the operations that will carry it out.

The Command pattern lets toolkit objects make requests of unspecified application objects by turning the request itself into an object. This object can be stored and passed around like other objects. The key to this pattern is an abstract Command class, which declares an interface for executing operations. In the simplest form this interface includes an abstract Execute operation. Concrete Command subclasses specify a receiver-action pair by storing the receiver as an instance variable and by implementing Execute to invoke the request. The receiver has the knowledge required to carry out the request.

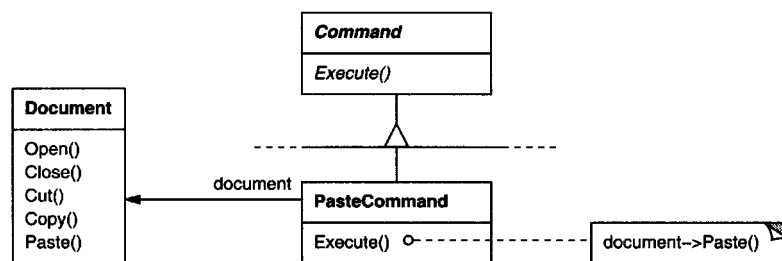


Menus can be implemented easily with Command objects. Each choice in a Menu is an instance of a MenuItem class. An Application class creates these menus and

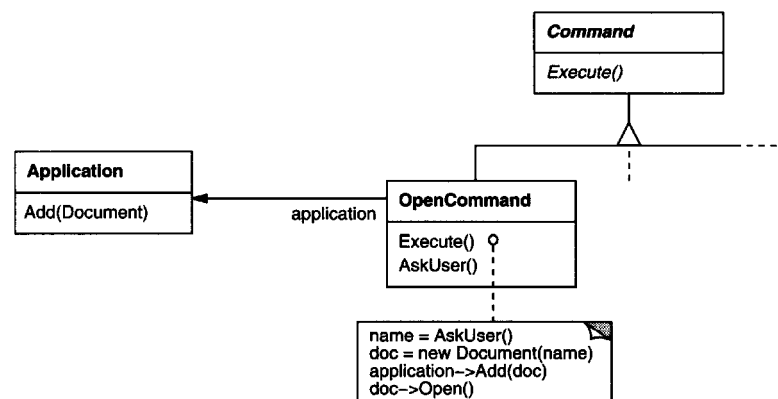
their menu items along with the rest of the user interface. The Application class also keeps track of Document objects that a user has opened.

The application configures each MenuItem with an instance of a concrete Command subclass. When the user selects a MenuItem, the MenuItem calls Execute on its command, and Execute carries out the operation. MenuItems don't know which subclass of Command they use. Command subclasses store the receiver of the request and invoke one or more operations on the receiver.

For example, PasteCommand supports pasting text from the clipboard into a Document. PasteCommand's receiver is the Document object it is supplied upon instantiation. The Execute operation invokes Paste on the receiving Document.

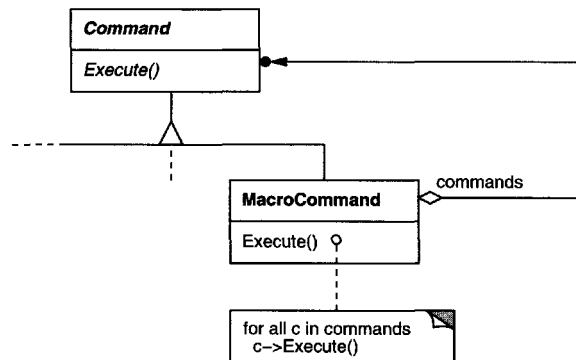


OpenCommand's Execute operation is different: it prompts the user for a document name, creates a corresponding Document object, adds the document to the receiving application, and opens the document.



Sometimes a MenuItem needs to execute a *sequence* of commands. For example, a MenuItem for centering a page at normal size could be constructed from a CenterDocumentCommand object and a NormalSizeCommand object. Because it's common to string commands together in this way, we can define a MacroCommand class to allow a MenuItem to execute an open-ended number of commands.

MacroCommand is a concrete Command subclass that simply executes a sequence of Commands. MacroCommand has no explicit receiver, because the commands it sequences define their own receiver.



In each of these examples, notice how the Command pattern decouples the object that invokes the operation from the one having the knowledge to perform it. This gives us a lot of flexibility in designing our user interface. An application can provide both a menu and a push button interface to a feature just by making the menu and the push button share an instance of the same concrete Command subclass. We can replace commands dynamically, which would be useful for implementing context-sensitive menus. We can also support command scripting by composing commands into larger ones. All of this is possible because the object that issues a request only needs to know how to issue it; it doesn't need to know how the request will be carried out.

Applicability

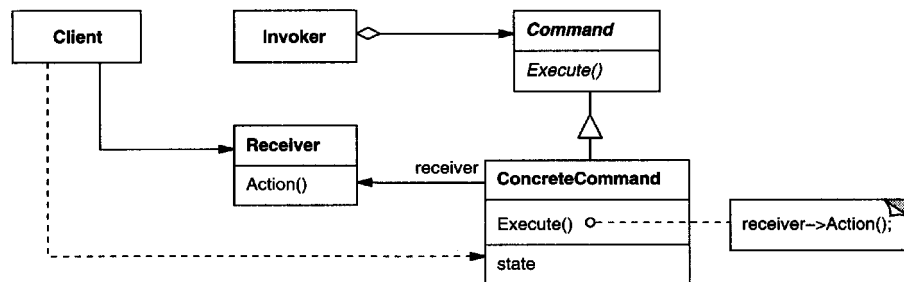
Use the Command pattern when you want to

- parameterize objects by an action to perform, as MenuItem objects did above. You can express such parameterization in a procedural language with a **callback** function, that is, a function that's registered somewhere to be called at a later point. Commands are an object-oriented replacement for callbacks.
- specify, queue, and execute requests at different times. A Command object can have a lifetime independent of the original request. If the receiver of a request can be represented in an address space-independent way, then you can transfer a command object for the request to a different process and fulfill the request there.
- support undo. The Command's Execute operation can store state for reversing its effects in the command itself. The Command interface must have an added Unexecute operation that reverses the effects of a previous call to Ex-

ecute. Executed commands are stored in a history list. Unlimited-level undo and redo is achieved by traversing this list backwards and forwards calling Unexecute and Execute, respectively.

- support logging changes so that they can be reapplied in case of a system crash. By augmenting the Command interface with load and store operations, you can keep a persistent log of changes. Recovering from a crash involves reloading logged commands from disk and reexecuting them with the Execute operation.
- structure a system around high-level operations built on primitives operations. Such a structure is common in information systems that support **transactions**. A transaction encapsulates a set of changes to data. The Command pattern offers a way to model transactions. Commands have a common interface, letting you invoke all transactions the same way. The pattern also makes it easy to extend the system with new transactions.

Structure



Participants

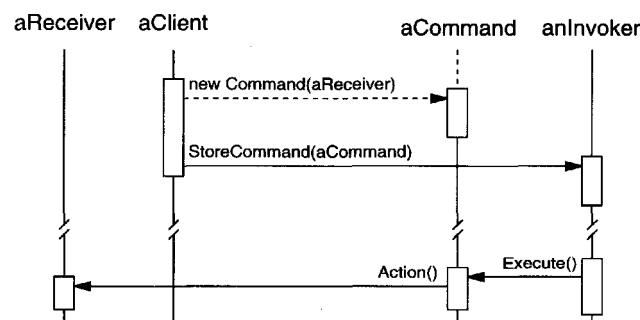
- **Command**
 - declares an interface for executing an operation.
- **ConcreteCommand** (PasteCommand, OpenCommand)
 - defines a binding between a Receiver object and an action.
 - implements Execute by invoking the corresponding operation(s) on Receiver.
- **Client** (Application)
 - creates a ConcreteCommand object and sets its receiver.
- **Invoker** (MenuItem)
 - asks the command to carry out the request.

- **Receiver** (Document, Application)
 - knows how to perform the operations associated with carrying out a request. Any class may serve as a Receiver.

Collaborations

- The client creates a ConcreteCommand object and specifies its receiver.
- An Invoker object stores the ConcreteCommand object.
- The invoker issues a request by calling `Execute` on the command. When commands are undoable, ConcreteCommand stores state for undoing the command prior to invoking `Execute`.
- The ConcreteCommand object invokes operations on its receiver to carry out the request.

The following diagram shows the interactions between these objects. It illustrates how Command decouples the invoker from the receiver (and the request it carries out).



Consequences

The Command pattern has the following consequences:

1. Command decouples the object that invokes the operation from the one that knows how to perform it.
2. Commands are first-class objects. They can be manipulated and extended like any other object.
3. You can assemble commands into a composite command. An example is the `MacroCommand` class described earlier. In general, composite commands are an instance of the Composite (163) pattern.
4. It's easy to add new Commands, because you don't have to change existing classes.

Implementation

Consider the following issues when implementing the Command pattern:

1. *How intelligent should a command be?* A command can have a wide range of abilities. At one extreme it merely defines a binding between a receiver and the actions that carry out the request. At the other extreme it implements everything itself without delegating to a receiver at all. The latter extreme is useful when you want to define commands that are independent of existing classes, when no suitable receiver exists, or when a command knows its receiver implicitly. For example, a command that creates another application window may be just as capable of creating the window as any other object. Somewhere in between these extremes are commands that have enough knowledge to find their receiver dynamically.
2. *Supporting undo and redo.* Commands can support undo and redo capabilities if they provide a way to reverse their execution (e.g., an `Unexecute` or `Undo` operation). A `ConcreteCommand` class might need to store additional state to do so. This state can include
 - the Receiver object, which actually carries out operations in response to the request,
 - the arguments to the operation performed on the receiver, and
 - any original values in the receiver that can change as a result of handling the request. The receiver must provide operations that let the command return the receiver to its prior state.

To support one level of undo, an application needs to store only the command that was executed last. For multiple-level undo and redo, the application needs a **history list** of commands that have been executed, where the maximum length of the list determines the number of undo/redo levels. The history list stores sequences of commands that have been executed. Traversing backward through the list and reverse-executing commands cancels their effect; traversing forward and executing commands reexecutes them.

An undoable command might have to be copied before it can be placed on the history list. That's because the command object that carried out the original request, say, from a `MenuItem`, will perform other requests at later times. Copying is required to distinguish different invocations of the same command if its state can vary across invocations.

For example, a `DeleteCommand` that deletes selected objects must store different sets of objects each time it's executed. Therefore the `DeleteCommand` object must be copied following execution, and the copy is placed on the history list. If the command's state never changes on execution, then copying is not required—only a reference to the command need be placed on the history list. Commands that must be copied before being placed on the history list act as prototypes (see Prototype (117)).

3. *Avoiding error accumulation in the undo process.* Hysteresis can be a problem in ensuring a reliable, semantics-preserving undo/redo mechanism. Errors can accumulate as commands are executed, unexecuted, and reexecuted repeatedly so that an application's state eventually diverges from original values. It may be necessary therefore to store more information in the command to ensure that objects are restored to their original state. The Memento (283) pattern can be applied to give the command access to this information without exposing the internals of other objects.
4. *Using C++ templates.* For commands that (1) aren't undoable and (2) don't require arguments, we can use C++ templates to avoid creating a Command subclass for every kind of action and receiver. We show how to do this in the Sample Code section.

Sample Code

The C++ code shown here sketches the implementation of the Command classes in the Motivation section. We'll define `OpenCommand`, `PasteCommand`, and `MacroCommand`. First the abstract Command class:

```
class Command {
public:
    virtual ~Command();

    virtual void Execute() = 0;
protected:
    Command();
};
```

`OpenCommand` opens a document whose name is supplied by the user. An `OpenCommand` must be passed an `Application` object in its constructor. `AskUser` is an implementation routine that prompts the user for the name of the document to open.

```
class OpenCommand : public Command {
public:
    OpenCommand(Application*);

    virtual void Execute();
protected:
    virtual const char* AskUser();
private:
    Application* _application;
    char* _response;
};

OpenCommand::OpenCommand (Application* a) {
    _application = a;
}
```

```

void OpenCommand::Execute () {
    const char* name = AskUser();

    if (name != 0) {
        Document* document = new Document(name);
        _application->Add(document);
        document->Open();
    }
}

```

A PasteCommand must be passed a Document object as its receiver. The receiver is given as a parameter to PasteCommand's constructor.

```

class PasteCommand : public Command {
public:
    PasteCommand(Document*);

    virtual void Execute();
private:
    Document* _document;
};

PasteCommand::PasteCommand (Document* doc) {
    _document = doc;
}

void PasteCommand::Execute () {
    _document->Paste();
}

```

For simple commands that aren't undoable and don't require arguments, we can use a class template to parameterize the command's receiver. We'll define a template subclass SimpleCommand for such commands. SimpleCommand is parameterized by the Receiver type and maintains a binding between a receiver object and an action stored as a pointer to a member function.

```

template <class Receiver>
class SimpleCommand : public Command {
public:
    typedef void (Receiver::* Action)();

    SimpleCommand(Receiver* r, Action a) :
        _receiver(r), _action(a) { }

    virtual void Execute();
private:
    Action _action;
    Receiver* _receiver;
};

```

The constructor stores the receiver and the action in the corresponding instance variables. Execute simply applies the action to the receiver.

```

template <class Receiver>
void SimpleCommand<Receiver>::Execute () {
    (_receiver->*_action)();
}

```

To create a command that calls `Action` on an instance of class `MyClass`, a client simply writes

```

MyClass* receiver = new MyClass;
// ...
Command* aCommand =
    new SimpleCommand<MyClass>(receiver, &MyClass::Action);
// ...
aCommand->Execute();

```

Keep in mind that this solution only works for simple commands. More complex commands that keep track of not only their receivers but also arguments and/or undo state require a `Command` subclass.

A `MacroCommand` manages a sequence of subcommands and provides operations for adding and removing subcommands. No explicit receiver is required, because the subcommands already define their receiver.

```

class MacroCommand : public Command {
public:
    MacroCommand();
    virtual ~MacroCommand();

    virtual void Add(Command*);
    virtual void Remove(Command*);

    virtual void Execute();
private:
    List<Command*>* _cmds;
};

```

The key to the `MacroCommand` is its `Execute` member function. This traverses all the subcommands and performs `Execute` on each of them.

```

void MacroCommand::Execute () {
    ListIterator<Command*> i(_cmds);

    for (i.First(); !i.IsDone(); i.Next()) {
        Command* c = i.CurrentItem();
        c->Execute();
    }
}

```

Note that should the `MacroCommand` implement an `Unexecute` operation, then its subcommands must be unexecuted in *reverse* order relative to `Execute`'s implementation.

Finally, `MacroCommand` must provide operations to manage its subcommands. The `MacroCommand` is also responsible for deleting its subcommands.

```
void MacroCommand::Add (Command* c) {  
    _cmds->Append(c);  
}  
  
void MacroCommand::Remove (Command* c) {  
    _cmds->Remove(c);  
}
```

Known Uses

Perhaps the first example of the Command pattern appears in a paper by Lieberman [Lie85]. MacApp [App89] popularized the notion of commands for implementing undoable operations. ET++ [WGM88], InterViews [LCI⁺92], and Unidraw [VL90] also define classes that follow the Command pattern. InterViews defines an Action abstract class that provides command functionality. It also defines an ActionCallback template, parameterized by action method, that can instantiate command subclasses automatically.

The THINK class library [Sym93b] also uses commands to support undoable actions. Commands in THINK are called “Tasks.” Task objects are passed along a Chain of Responsibility (223) for consumption.

Unidraw’s command objects are unique in that they can behave like messages. A Unidraw command may be sent to another object for interpretation, and the result of the interpretation varies with the receiving object. Moreover, the receiver may delegate the interpretation to another object, typically the receiver’s parent in a larger structure as in a Chain of Responsibility. The receiver of a Unidraw command is thus computed rather than stored. Unidraw’s interpretation mechanism depends on run-time type information.

Coplien describes how to implement **functors**, objects that are functions, in C++ [Cop92]. He achieves a degree of transparency in their use by overloading the function call operator (`operator()`). The Command pattern is different; its focus is on maintaining a *binding between* a receiver and a function (i.e., action), not just maintaining a function.

Related Patterns

A Composite (163) can be used to implement MacroCommands.

A Memento (283) can keep state the command requires to undo its effect.

A command that must be copied before being placed on the history list acts as a Prototype (117).

COMPOSITE
Object Structural

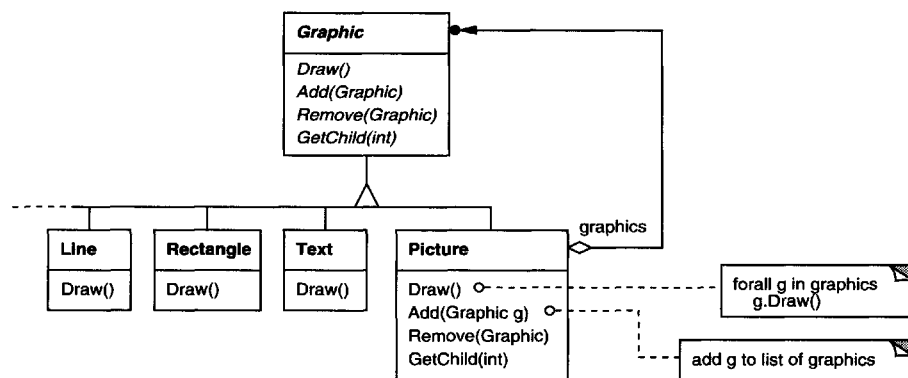
Intent

Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.

Motivation

Graphics applications like drawing editors and schematic capture systems let users build complex diagrams out of simple components. The user can group components to form larger components, which in turn can be grouped to form still larger components. A simple implementation could define classes for graphical primitives such as Text and Lines plus other classes that act as containers for these primitives.

But there's a problem with this approach: Code that uses these classes must treat primitive and container objects differently, even if most of the time the user treats them identically. Having to distinguish these objects makes the application more complex. The Composite pattern describes how to use recursive composition so that clients don't have to make this distinction.

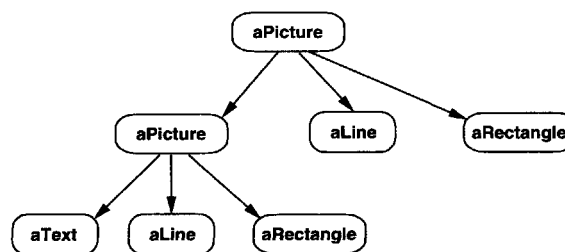


The key to the Composite pattern is an abstract class that represents *both* primitives and their containers. For the graphics system, this class is **Graphic**. **Graphic** declares operations like `Draw` that are specific to graphical objects. It also declares operations that all composite objects share, such as operations for accessing and managing its children.

The subclasses *Line*, *Rectangle*, and *Text* (see preceding class diagram) define primitive graphical objects. These classes implement *Draw* to draw lines, rectangles, and text, respectively. Since primitive graphics have no child graphics, none of these subclasses implements child-related operations.

The *Picture* class defines an aggregate of *Graphic* objects. *Picture* implements *Draw* to call *Draw* on its children, and it implements child-related operations accordingly. Because the *Picture* interface conforms to the *Graphic* interface, *Picture* objects can compose other *Pictures* recursively.

The following diagram shows a typical composite object structure of recursively composed *Graphic* objects:

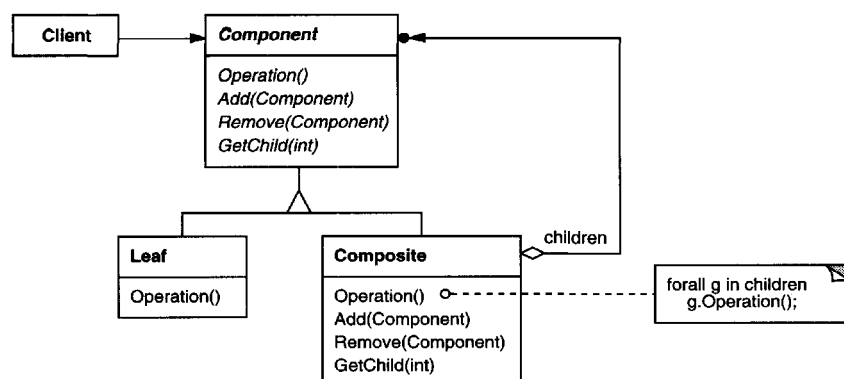


Applicability

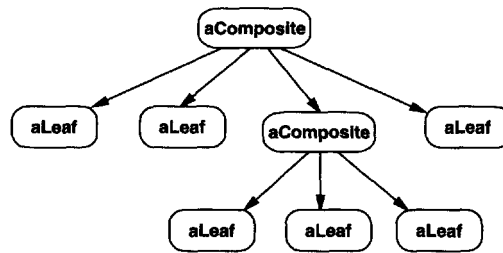
Use the Composite pattern when

- you want to represent part-whole hierarchies of objects.
- you want clients to be able to ignore the difference between compositions of objects and individual objects. Clients will treat all objects in the composite structure uniformly.

Structure



A typical Composite object structure might look like this:



Participants

- **Component** (Graphic)
 - declares the interface for objects in the composition.
 - implements default behavior for the interface common to all classes, as appropriate.
 - declares an interface for accessing and managing its child components.
 - (optional) defines an interface for accessing a component's parent in the recursive structure, and implements it if that's appropriate.
- **Leaf** (Rectangle, Line, Text, etc.)
 - represents leaf objects in the composition. A leaf has no children.
 - defines behavior for primitive objects in the composition.
- **Composite** (Picture)
 - defines behavior for components having children.
 - stores child components.
 - implements child-related operations in the Component interface.
- **Client**
 - manipulates objects in the composition through the Component interface.

Collaborations

- Clients use the Component class interface to interact with objects in the composite structure. If the recipient is a Leaf, then the request is handled directly. If the recipient is a Composite, then it usually forwards requests to its child components, possibly performing additional operations before and/or after forwarding.

Consequences

The Composite pattern

- defines class hierarchies consisting of primitive objects and composite objects. Primitive objects can be composed into more complex objects, which in turn can be composed, and so on recursively. Wherever client code expects a primitive object, it can also take a composite object.
- makes the client simple. Clients can treat composite structures and individual objects uniformly. Clients normally don't know (and shouldn't care) whether they're dealing with a leaf or a composite component. This simplifies client code, because it avoids having to write tag-and-case-statement-style functions over the classes that define the composition.
- makes it easier to add new kinds of components. Newly defined Composite or Leaf subclasses work automatically with existing structures and client code. Clients don't have to be changed for new Component classes.
- can make your design overly general. The disadvantage of making it easy to add new components is that it makes it harder to restrict the components of a composite. Sometimes you want a composite to have only certain components. With Composite, you can't rely on the type system to enforce those constraints for you. You'll have to use run-time checks instead.

Implementation

There are many issues to consider when implementing the Composite pattern:

1. *Explicit parent references.* Maintaining references from child components to their parent can simplify the traversal and management of a composite structure. The parent reference simplifies moving up the structure and deleting a component. Parent references also help support the Chain of Responsibility (223) pattern.

The usual place to define the parent reference is in the Component class. Leaf and Composite classes can inherit the reference and the operations that manage it.

With parent references, it's essential to maintain the invariant that all children of a composite have as their parent the composite that in turn has them as children. The easiest way to ensure this is to change a component's parent *only* when it's being added or removed from a composite. If this can be implemented once in the Add and Remove operations of the Composite class, then it can be inherited by all the subclasses, and the invariant will be maintained automatically.

2. *Sharing components.* It's often useful to share components, for example, to reduce storage requirements. But when a component can have no more than one parent, sharing components becomes difficult.

A possible solution is for children to store multiple parents. But that can lead to ambiguities as a request propagates up the structure. The Flyweight (195) pattern shows how to rework a design to avoid storing parents altogether. It works in cases where children can avoid sending parent requests by externalizing some or all of their state.

3. *Maximizing the Component interface.* One of the goals of the Composite pattern is to make clients unaware of the specific Leaf or Composite classes they're using. To attain this goal, the Component class should define as many common operations for Composite and Leaf classes as possible. The Component class usually provides default implementations for these operations, and Leaf and Composite subclasses will override them.

However, this goal will sometimes conflict with the principle of class hierarchy design that says a class should only define operations that are meaningful to its subclasses. There are many operations that Component supports that don't seem to make sense for Leaf classes. How can Component provide a default implementation for them?

Sometimes a little creativity shows how an operation that would appear to make sense only for Composites can be implemented for all Components by moving it to the Component class. For example, the interface for accessing children is a fundamental part of a Composite class but not necessarily Leaf classes. But if we view a Leaf as a Component that *never* has children, then we can define a default operation for child access in the Component class that *never returns* any children. Leaf classes can use the default implementation, but Composite classes will reimplement it to return their children.

The child management operations are more troublesome and are discussed in the next item.

4. *Declaring the child management operations.* Although the Composite class *implements* the Add and Remove operations for managing children, an important issue in the Composite pattern is which classes *declare* these operations in the Composite class hierarchy. Should we declare these operations in the Component and make them meaningful for Leaf classes, or should we declare and define them only in Composite and its subclasses?

The decision involves a trade-off between safety and transparency:

- Defining the child management interface at the root of the class hierarchy gives you transparency, because you can treat all components uniformly. It costs you safety, however, because clients may try to do meaningless things like add and remove objects from leaves.
- Defining child management in the Composite class gives you safety, because any attempt to add or remove objects from leaves will be caught at compile-time in a statically typed language like C++. But you lose transparency, because leaves and composites have different interfaces.

We have emphasized transparency over safety in this pattern. If you opt for safety, then at times you may lose type information and have to convert a component into a composite. How can you do this without resorting to a type-unsafe cast?

One approach is to declare an operation `Composite* GetComponent()` in the `Component` class. `Component` provides a default operation that returns a null pointer. The `Composite` class redefines this operation to return itself through the `this` pointer:

```
class Composite;

class Component {
public:
    //...
    virtual Composite* GetComponent() { return 0; }
};

class Composite : public Component {
public:
    void Add(Component*);
    // ...
    virtual Composite* GetComponent() { return this; }
};

class Leaf : public Component {
    // ...
};
```

`GetComponent` lets you query a component to see if it's a composite. You can perform `Add` and `Remove` safely on the composite it returns.

```
Composite* aComposite = new Composite;
Leaf* aLeaf = new Leaf;

Component* aComponent;
Composite* test;

aComponent = aComposite;
if (test = aComponent->GetComponent()) {
    test->Add(new Leaf);
}

aComponent = aLeaf;

if (test = aComponent->GetComponent()) {
    test->Add(new Leaf); // will not add leaf
}
```

Similar tests for a `Composite` can be done using the C++ `dynamic_cast` construct.

Of course, the problem here is that we don't treat all components uniformly. We have to revert to testing for different types before taking the appropriate action.

The only way to provide transparency is to define default `Add` and `Remove` operations in `Component`. That creates a new problem: There's no way to implement `Component : : Add` without introducing the possibility of it failing. You could make it do nothing, but that ignores an important consideration; that is, an attempt to add something to a leaf probably indicates a bug. In that case, the `Add` operation produces garbage. You could make it delete its argument, but that might not be what clients expect.

Usually it's better to make `Add` and `Remove` fail by default (perhaps by raising an exception) if the component isn't allowed to have children or if the argument of `Remove` isn't a child of the component, respectively.

Another alternative is to change the meaning of "remove" slightly. If the component maintains a parent reference, then we could redefine `Component : : Remove` to remove itself from its parent. However, there still isn't a meaningful interpretation for a corresponding `Add`.

5. *Should Component implement a list of Components?* You might be tempted to define the set of children as an instance variable in the `Component` class where the child access and management operations are declared. But putting the child pointer in the base class incurs a space penalty for every leaf, even though a leaf never has children. This is worthwhile only if there are relatively few children in the structure.
6. *Child ordering.* Many designs specify an ordering on the children of `Composite`. In the earlier `Graphics` example, ordering may reflect front-to-back ordering. If `Composites` represent parse trees, then compound statements can be instances of a `Composite` whose children must be ordered to reflect the program.
When child ordering is an issue, you must design child access and management interfaces carefully to manage the sequence of children. The `Iterator` (257) pattern can guide you in this.
7. *Caching to improve performance.* If you need to traverse or search compositions frequently, the `Composite` class can cache traversal or search information about its children. The `Composite` can cache actual results or just information that lets it short-circuit the traversal or search. For example, the `Picture` class from the `Motivation` example could cache the bounding box of its children. During drawing or selection, this cached bounding box lets the `Picture` avoid drawing or searching when its children aren't visible in the current window. Changes to a component will require invalidating the caches of its parents. This works best when components know their parents. So if you're using caching, you need to define an interface for telling composites that their caches are invalid.
8. *Who should delete components?* In languages without garbage collection, it's usually best to make a `Composite` responsible for deleting its children when it's destroyed. An exception to this rule is when `Leaf` objects are immutable and thus can be shared.

9. *What's the best data structure for storing components?* Composites may use a variety of data structures to store their children, including linked lists, trees, arrays, and hash tables. The choice of data structure depends (as always) on efficiency. In fact, it isn't even necessary to use a general-purpose data structure at all. Sometimes composites have a variable for each child, although this requires each subclass of Composite to implement its own management interface. See Interpreter (243) for an example.

Sample Code

Equipment such as computers and stereo components are often organized into part-whole or containment hierarchies. For example, a chassis can contain drives and planar boards, a bus can contain cards, and a cabinet can contain chassis, buses, and so forth. Such structures can be modeled naturally with the Composite pattern.

Equipment class defines an interface for all equipment in the part-whole hierarchy.

```
class Equipment {
public:
    virtual ~Equipment();

    const char* Name() { return _name; }

    virtual Watt Power();
    virtual Currency NetPrice();
    virtual Currency DiscountPrice();

    virtual void Add(Equipment*);
    virtual void Remove(Equipment*);
    virtual Iterator<Equipment*>* CreateIterator();
protected:
    Equipment(const char*);
private:
    const char* _name;
};
```

Equipment declares operations that return the attributes of a piece of equipment, like its power consumption and cost. Subclasses implement these operations for specific kinds of equipment. Equipment also declares a `CreateIterator` operation that returns an `Iterator` (see Appendix C) for accessing its parts. The default implementation for this operation returns a `NullIterator`, which iterates over the empty set.

Subclasses of Equipment might include Leaf classes that represent disk drives, integrated circuits, and switches:

```

class FloppyDisk : public Equipment {
public:
    FloppyDisk(const char*);
    virtual ~FloppyDisk();

    virtual Watt Power();
    virtual Currency NetPrice();
    virtual Currency DiscountPrice();
};

```

CompositeEquipment is the base class for equipment that contains other equipment. It's also a subclass of Equipment.

```

class CompositeEquipment : public Equipment {
public:
    virtual ~CompositeEquipment();

    virtual Watt Power();
    virtual Currency NetPrice();
    virtual Currency DiscountPrice();

    virtual void Add(Equipment*);
    virtual void Remove(Equipment*);
    virtual Iterator<Equipment*>* CreateIterator();

protected:
    CompositeEquipment(const char*);
private:
    List<Equipment*> _equipment;
};

```

CompositeEquipment defines the operations for accessing and managing subequipment. The operations Add and Remove insert and delete equipment from the list of equipment stored in the `_equipment` member. The operation CreateIterator returns an iterator (specifically, an instance of ListIterator) that will traverse this list.

A default implementation of NetPrice might use CreateIterator to sum the net prices of the subequipment²:

```

Currency CompositeEquipment::NetPrice () {
    Iterator<Equipment*>* i = CreateIterator();
    Currency total = 0;

    for (i->First(); !i->IsDone(); i->Next()) {
        total += i->CurrentItem()->NetPrice();
    }
    delete i;
    return total;
}

```

²It's easy to forget to delete the iterator once you're done with it. The Iterator pattern shows how to guard against such bugs on page 266.

Now we can represent a computer chassis as a subclass of `CompositeEquipment` called `Chassis`. `Chassis` inherits the child-related operations from `CompositeEquipment`.

```
class Chassis : public CompositeEquipment {
public:
    Chassis(const char*);
    virtual ~Chassis();

    virtual Watt Power();
    virtual Currency NetPrice();
    virtual Currency DiscountPrice();
};
```

We can define other equipment containers such as `Cabinet` and `Bus` in a similar way. That gives us everything we need to assemble equipment into a (pretty simple) personal computer:

```
Cabinet* cabinet = new Cabinet("PC Cabinet");
Chassis* chassis = new Chassis("PC Chassis");

cabinet->Add(chassis);

Bus* bus = new Bus("MCA Bus");
bus->Add(new Card("16Mbs Token Ring"));

chassis->Add(bus);
chassis->Add(new FloppyDisk("3.5in Floppy"));

cout << "The net price is " << chassis->NetPrice() << endl;
```

Known Uses

Examples of the Composite pattern can be found in almost all object-oriented systems. The original `View` class of Smalltalk Model/View/Controller [KP88] was a `Composite`, and nearly every user interface toolkit or framework has followed in its steps, including ET++ (with its `VObjects` [WGM88]) and `InterViews` (`Styles` [LCI⁺92], `Graphics` [VL88], and `Glyphs` [CL90]). It's interesting to note that the original `View` of Model/View/Controller had a set of subviews; in other words, `View` was both the `Component` class and the `Composite` class. Release 4.0 of Smalltalk-80 revised Model/View/Controller with a `VisualComponent` class that has subclasses `View` and `CompositeView`.

The RTL Smalltalk compiler framework [JML92] uses the Composite pattern extensively. `RTLExpression` is a `Component` class for parse trees. It has subclasses, such as `BinaryExpression`, that contain child `RTLExpression` objects. These classes define a composite structure for parse trees. `RegisterTransfer` is the `Component` class for a program's intermediate Single Static Assignment (SSA) form. Leaf subclasses of `RegisterTransfer` define different static assignments such as

- primitive assignments that perform an operation on two registers and assign the result to a third;
- an assignment with a source register but no destination register, which indicates that the register is used after a routine returns; and
- an assignment with a destination register but no source, which indicates that the register is assigned before the routine starts.

Another subclass, *RegisterTransferSet*, is a Composite class for representing assignments that change several registers at once.

Another example of this pattern occurs in the financial domain, where a portfolio aggregates individual assets. You can support complex aggregations of assets by implementing a portfolio as a Composite that conforms to the interface of an individual asset [BE93].

The Command (233) pattern describes how Command objects can be composed and sequenced with a *MacroCommand* Composite class.

Related Patterns

Often the component-parent link is used for a Chain of Responsibility (223).

Decorator (175) is often used with Composite. When decorators and composites are used together, they will usually have a common parent class. So decorators will have to support the Component interface with operations like *Add*, *Remove*, and *GetChild*.

Flyweight (195) lets you share components, but they can no longer refer to their parents.

Iterator (257) can be used to traverse composites.

Visitor (331) localizes operations and behavior that would otherwise be distributed across Composite and Leaf classes.

DECORATOR

Object Structural

Intent

Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

Also Known As

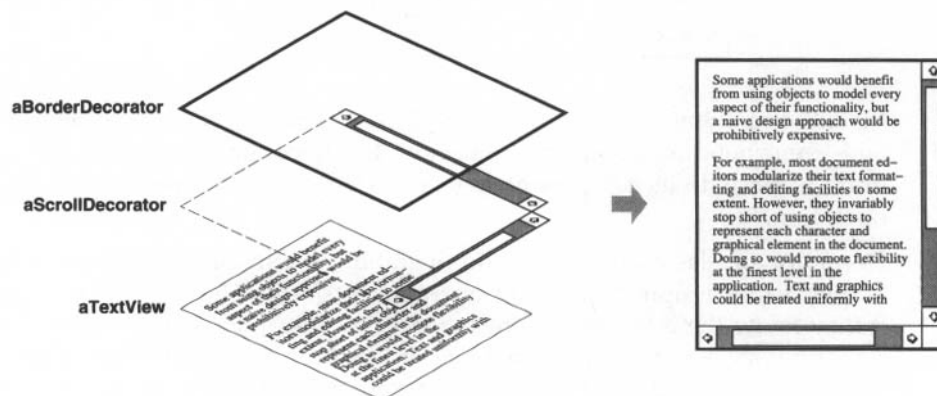
Wrapper

Motivation

Sometimes we want to add responsibilities to individual objects, not to an entire class. A graphical user interface toolkit, for example, should let you add properties like borders or behaviors like scrolling to any user interface component.

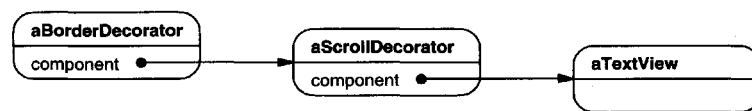
One way to add responsibilities is with inheritance. Inheriting a border from another class puts a border around every subclass instance. This is inflexible, however, because the choice of border is made statically. A client can't control how and when to decorate the component with a border.

A more flexible approach is to enclose the component in another object that adds the border. The enclosing object is called a **decorator**. The decorator conforms to the interface of the component it decorates so that its presence is transparent to the component's clients. The decorator forwards requests to the component and may perform additional actions (such as drawing a border) before or after forwarding. Transparency lets you nest decorators recursively, thereby allowing an unlimited number of added responsibilities.

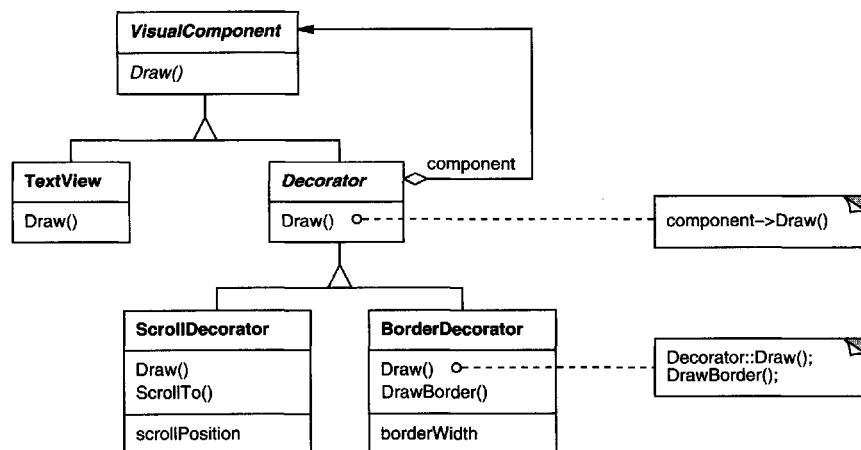


For example, suppose we have a `TextView` object that displays text in a window. `TextView` has no scroll bars by default, because we might not always need them. When we do, we can use a `ScrollDecorator` to add them. Suppose we also want to add a thick black border around the `TextView`. We can use a `BorderDecorator` to add this as well. We simply compose the decorators with the `TextView` to produce the desired result.

The following object diagram shows how to compose a `TextView` object with `BorderDecorator` and `ScrollDecorator` objects to produce a bordered, scrollable text view:



The `ScrollDecorator` and `BorderDecorator` classes are subclasses of `Decorator`, an abstract class for visual components that decorate other visual components.



`VisualComponent` is the abstract class for visual objects. It defines their drawing and event handling interface. Note how the `Decorator` class simply forwards draw requests to its component, and how `Decorator` subclasses can extend this operation.

`Decorator` subclasses are free to add operations for specific functionality. For example, `ScrollDecorator`'s `ScrollTo` operation lets other objects scroll the interface *if* they know there happens to be a `ScrollDecorator` object in the interface. The important aspect of this pattern is that it lets decorators appear anywhere a `VisualComponent` can. That way clients generally can't tell the difference between

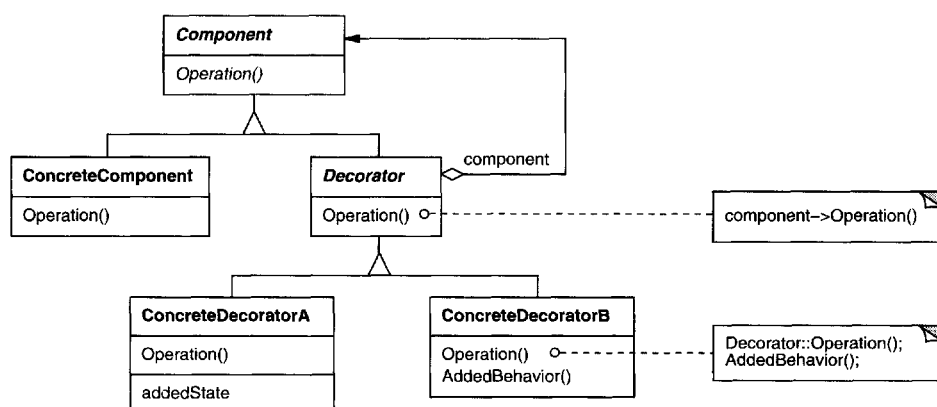
a decorated component and an undecorated one, and so they don't depend at all on the decoration.

Applicability

Use Decorator

- to add responsibilities to individual objects dynamically and transparently, that is, without affecting other objects.
- for responsibilities that can be withdrawn.
- when extension by subclassing is impractical. Sometimes a large number of independent extensions are possible and would produce an explosion of subclasses to support every combination. Or a class definition may be hidden or otherwise unavailable for subclassing.

Structure



Participants

- **Component** (`VisualComponent`)
 - defines the interface for objects that can have responsibilities added to them dynamically.
- **ConcreteComponent** (`TextView`)
 - defines an object to which additional responsibilities can be attached.
- **Decorator**
 - maintains a reference to a **Component** object and defines an interface that conforms to **Component**'s interface.

- **ConcreteDecorator** (BorderDecorator, ScrollDecorator)
 - adds responsibilities to the component.

Collaborations

- Decorator forwards requests to its Component object. It may optionally perform additional operations before and after forwarding the request.

Consequences

The Decorator pattern has at least two key benefits and two liabilities:

1. *More flexibility than static inheritance.* The Decorator pattern provides a more flexible way to add responsibilities to objects than can be had with static (multiple) inheritance. With decorators, responsibilities can be added and removed at run-time simply by attaching and detaching them. In contrast, inheritance requires creating a new class for each additional responsibility (e.g., BorderedScrollableTextView, BorderedTextView). This gives rise to many classes and increases the complexity of a system. Furthermore, providing different Decorator classes for a specific Component class lets you mix and match responsibilities.

Decorators also make it easy to add a property twice. For example, to give a TextView a double border, simply attach two BorderDecorators. Inheriting from a Border class twice is error-prone at best.
2. *Avoids feature-laden classes high up in the hierarchy.* Decorator offers a pay-as-you-go approach to adding responsibilities. Instead of trying to support all foreseeable features in a complex, customizable class, you can define a simple class and add functionality incrementally with Decorator objects. Functionality can be composed from simple pieces. As a result, an application needn't pay for features it doesn't use. It's also easy to define new kinds of Decorators independently from the classes of objects they extend, even for unforeseen extensions. Extending a complex class tends to expose details unrelated to the responsibilities you're adding.
3. *A decorator and its component aren't identical.* A decorator acts as a transparent enclosure. But from an object identity point of view, a decorated component is not identical to the component itself. Hence you shouldn't rely on object identity when you use decorators.
4. *Lots of little objects.* A design that uses Decorator often results in systems composed of lots of little objects that all look alike. The objects differ only in the way they are interconnected, not in their class or in the value of their variables. Although these systems are easy to customize by those who understand them, they can be hard to learn and debug.

Implementation

Several issues should be considered when applying the Decorator pattern:

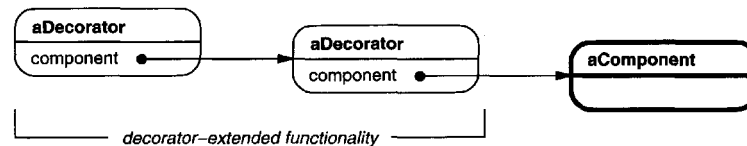
1. *Interface conformance.* A decorator object's interface must conform to the interface of the component it decorates. ConcreteDecorator classes must therefore inherit from a common class (at least in C++).
2. *Omitting the abstract Decorator class.* There's no need to define an abstract Decorator class when you only need to add one responsibility. That's often the case when you're dealing with an existing class hierarchy rather than designing a new one. In that case, you can merge Decorator's responsibility for forwarding requests to the component into the ConcreteDecorator.
3. *Keeping Component classes lightweight.* To ensure a conforming interface, components and decorators must descend from a common Component class. It's important to keep this common class lightweight; that is, it should focus on defining an interface, not on storing data. The definition of the data representation should be deferred to subclasses; otherwise the complexity of the Component class might make the decorators too heavyweight to use in quantity. Putting a lot of functionality into Component also increases the probability that concrete subclasses will pay for features they don't need.
4. *Changing the skin of an object versus changing its guts.* We can think of a decorator as a skin over an object that changes its behavior. An alternative is to change the object's guts. The Strategy (315) pattern is a good example of a pattern for changing the guts.

Strategies are a better choice in situations where the Component class is intrinsically heavyweight, thereby making the Decorator pattern too costly to apply. In the Strategy pattern, the component forwards some of its behavior to a separate strategy object. The Strategy pattern lets us alter or extend the component's functionality by replacing the strategy object.

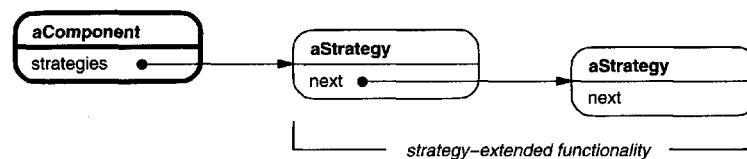
For example, we can support different border styles by having the component defer border-drawing to a separate Border object. The Border object is a Strategy object that encapsulates a border-drawing strategy. By extending the number of strategies from just one to an open-ended list, we achieve the same effect as nesting decorators recursively.

In MacApp 3.0 [App89] and Bedrock [Sym93a], for example, graphical components (called "views") maintain a list of "adorners" objects that can attach additional adornments like borders to a view component. If a view has any adorners attached, then it gives them a chance to draw additional embellishments. MacApp and Bedrock must use this approach because the View class is heavyweight. It would be too expensive to use a full-fledged View just to add a border.

Since the Decorator pattern only changes a component from the outside, the component doesn't have to know anything about its decorators; that is, the decorators are transparent to the component:



With strategies, the component itself knows about possible extensions. So it has to reference and maintain the corresponding strategies:



The Strategy-based approach might require modifying the component to accommodate new extensions. On the other hand, a strategy can have its own specialized interface, whereas a decorator's interface must conform to the component's. A strategy for rendering a border, for example, need only define the interface for rendering a border (`DrawBorder`, `GetWidth`, etc.), which means that the strategy can be lightweight even if the Component class is heavyweight.

MacApp and Bedrock use this approach for more than just adorning views. They also use it to augment the event-handling behavior of objects. In both systems, a view maintains a list of "behavior" objects that can modify and intercept events. The view gives each of the registered behavior objects a chance to handle the event before nonregistered behaviors, effectively overriding them. You can decorate a view with special keyboard-handling support, for example, by registering a behavior object that intercepts and handles key events.

Sample Code

The following code shows how to implement user interface decorators in C++. We'll assume there's a Component class called `VisualComponent`.

```

class VisualComponent {
public:
    VisualComponent();

    virtual void Draw();
    virtual void Resize();
    // ...
};

```

We define a subclass of `VisualComponent` called `Decorator`, which we'll subclass to obtain different decorations.

```
class Decorator : public VisualComponent {
public:
    Decorator(VisualComponent*);

    virtual void Draw();
    virtual void Resize();
    // ...
private:
    VisualComponent* _component;
};
```

`Decorator` decorates the `VisualComponent` referenced by the `_component` instance variable, which is initialized in the constructor. For each operation in `VisualComponent`'s interface, `Decorator` defines a default implementation that passes the request on to `_component`:

```
void Decorator::Draw () {
    _component->Draw();
}

void Decorator::Resize () {
    _component->Resize();
}
```

Subclasses of `Decorator` define specific decorations. For example, the class `BorderDecorator` adds a border to its enclosing component. `BorderDecorator` is a subclass of `Decorator` that overrides the `Draw` operation to draw the border. `BorderDecorator` also defines a private `DrawBorder` helper operation that does the drawing. The subclass inherits all other operation implementations from `Decorator`.

```
class BorderDecorator : public Decorator {
public:
    BorderDecorator(VisualComponent*, int borderWidth);

    virtual void Draw();
private:
    void DrawBorder(int);
private:
    int _width;
};

void BorderDecorator::Draw () {
    Decorator::Draw();
    DrawBorder(_width);
}
```

A similar implementation would follow for `ScrollDecorator` and `DropShadowDecorator`, which would add scrolling and drop shadow capabilities to a visual component.

Now we can compose instances of these classes to provide different decorations. The following code illustrates how we can use decorators to create a bordered scrollable `TextView`.

First, we need a way to put a visual component into a window object. We'll assume our `Window` class provides a `SetContents` operation for this purpose:

```
void Window::SetContents (VisualComponent* contents) {  
    // ...  
}
```

Now we can create the text view and a window to put it in:

```
Window* window = new Window;  
TextView* textView = new TextView;
```

`TextView` is a `VisualComponent`, which lets us put it into the window:

```
window->SetContents(textView);
```

But we want a bordered and scrollable `TextView`. So we decorate it accordingly before putting it in the window.

```
window->SetContents(  
    new BorderDecorator(  
        new ScrollDecorator(textView), 1  
    )  
);
```

Because `Window` accesses its contents through the `VisualComponent` interface, it's unaware of the decorator's presence. You, as the client, can still keep track of the text view if you have to interact with it directly, for example, when you need to invoke operations that aren't part of the `VisualComponent` interface. Clients that rely on the component's identity should refer to it directly as well.

Known Uses

Many object-oriented user interface toolkits use decorators to add graphical embellishments to widgets. Examples include `InterViews` [LVC89, LCI⁺92], `ET++` [WGM88], and the `ObjectWorks\Smalltalk` class library [Par90]. More exotic applications of `Decorator` are the `DebuggingGlyph` from `InterViews` and the `PassivityWrapper` from `ParcPlace Smalltalk`. A `DebuggingGlyph` prints out debugging information before and after it forwards a layout request to its component. This trace information can be used to analyze and debug the layout behavior

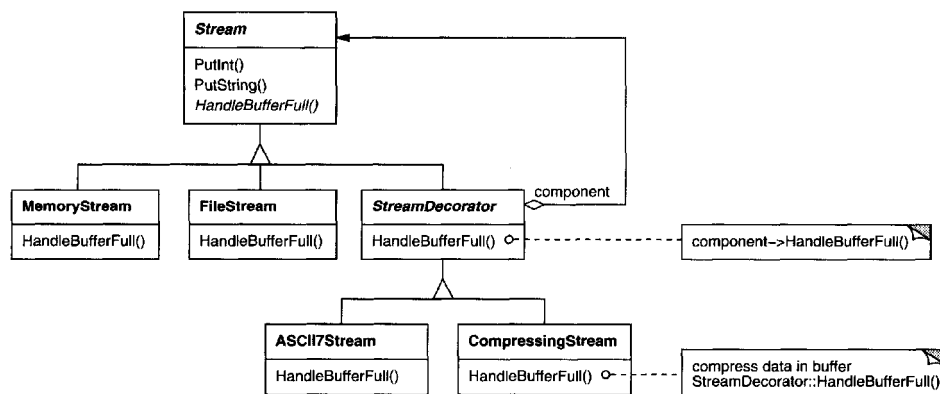
of objects in a complex composition. The PassivityWrapper can enable or disable user interactions with the component.

But the Decorator pattern is by no means limited to graphical user interfaces, as the following example (based on the ET++ streaming classes [WGM88]) illustrates.

Streams are a fundamental abstraction in most I/O facilities. A stream can provide an interface for converting objects into a sequence of bytes or characters. That lets us transcribe an object to a file or to a string in memory for retrieval later. A straightforward way to do this is to define an abstract Stream class with subclasses MemoryStream and FileStream. But suppose we also want to be able to do the following:

- Compress the stream data using different compression algorithms (run-length encoding, Lempel-Ziv, etc.).
- Reduce the stream data to 7-bit ASCII characters so that it can be transmitted over an ASCII communication channel.

The Decorator pattern gives us an elegant way to add these responsibilities to streams. The diagram below shows one solution to the problem:



The **Stream** abstract class maintains an internal buffer and provides operations for storing data onto the stream (`PutInt`, `PutString`). Whenever the buffer is full, **Stream** calls the abstract operation `HandleBufferFull`, which does the actual data transfer. The **FileStream** version of this operation overrides this operation to transfer the buffer to a file.

The key class here is **StreamDecorator**, which maintains a reference to a component stream and forwards requests to it. **StreamDecorator** subclasses override `HandleBufferFull` and perform additional actions before calling **StreamDecorator**'s `HandleBufferFull` operation.

For example, the `CompressingStream` subclass compresses the data, and the `ASCII7Stream` converts the data into 7-bit ASCII. Now, to create a `FileStream` that compresses its data *and* converts the compressed binary data to 7-bit ASCII, we decorate a `FileStream` with a `CompressingStream` and an `ASCII7Stream`:

```
Stream* aStream = new CompressingStream(  
    new ASCII7Stream(  
        new FileStream("aFileName")  
    )  
);  
aStream->PutInt(12);  
aStream->PutString("aString");
```

Related Patterns

Adapter (139): A decorator is different from an adapter in that a decorator only changes an object's responsibilities, not its interface; an adapter will give an object a completely new interface.

Composite (163): A decorator can be viewed as a degenerate composite with only one component. However, a decorator adds additional responsibilities—it isn't intended for object aggregation.

Strategy (315): A decorator lets you change the skin of an object; a strategy lets you change the guts. These are two alternative ways of changing an object.

ITERATOR

Object Behavioral

Intent

Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

Also Known As

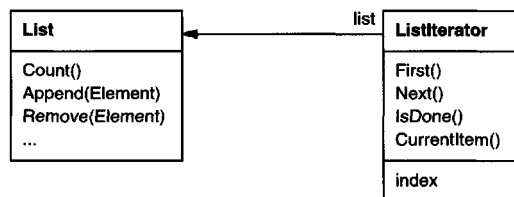
Cursor

Motivation

An aggregate object such as a list should give you a way to access its elements without exposing its internal structure. Moreover, you might want to traverse the list in different ways, depending on what you want to accomplish. But you probably don't want to bloat the List interface with operations for different traversals, even if you could anticipate the ones you will need. You might also need to have more than one traversal pending on the same list.

The Iterator pattern lets you do all this. The key idea in this pattern is to take the responsibility for access and traversal out of the list object and put it into an **iterator** object. The Iterator class defines an interface for accessing the list's elements. An iterator object is responsible for keeping track of the current element; that is, it knows which elements have been traversed already.

For example, a List class would call for a ListIterator with the following relationship between them:



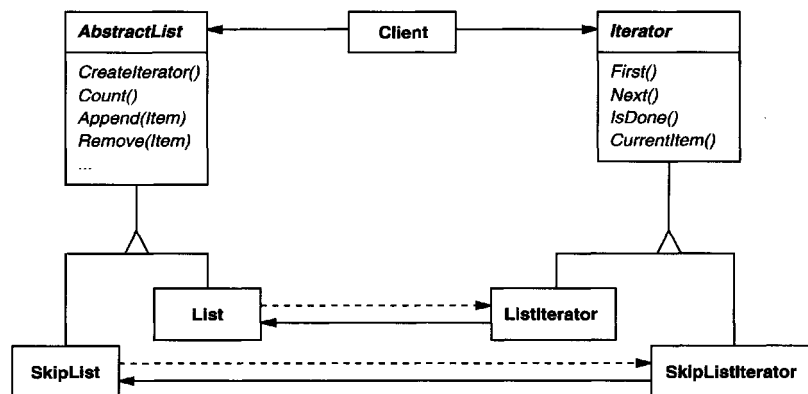
Before you can instantiate `ListIterator`, you must supply the `List` to traverse. Once you have the `ListIterator` instance, you can access the list's elements sequentially. The `CurrentItem` operation returns the current element in the list, `First` initializes the current element to the first element, `Next` advances the current element to the next element, and `IsDone` tests whether we've advanced beyond the last element—that is, we're finished with the traversal.

Separating the traversal mechanism from the List object lets us define iterators for different traversal policies without enumerating them in the List interface. For example, *FilteringListIterator* might provide access only to those elements that satisfy specific filtering constraints.

Notice that the iterator and the list are coupled, and the client must know that it is a *list* that's traversed as opposed to some other aggregate structure. Hence the client commits to a particular aggregate structure. It would be better if we could change the aggregate class without changing client code. We can do this by generalizing the iterator concept to support **polymorphic iteration**.

As an example, let's assume that we also have a *SkipList* implementation of a list. A *skiplist* [Pug90] is a probabilistic data structure with characteristics similar to balanced trees. We want to be able to write code that works for both *List* and *SkipList* objects.

We define an *AbstractList* class that provides a common interface for manipulating lists. Similarly, we need an abstract *Iterator* class that defines a common iteration interface. Then we can define concrete *Iterator* subclasses for the different list implementations. As a result, the iteration mechanism becomes independent of concrete aggregate classes.



The remaining problem is how to create the iterator. Since we want to write code that's independent of the concrete *List* subclasses, we cannot simply instantiate a specific class. Instead, we make the list objects responsible for creating their corresponding iterator. This requires an operation like *CreateIterator* through which clients request an iterator object.

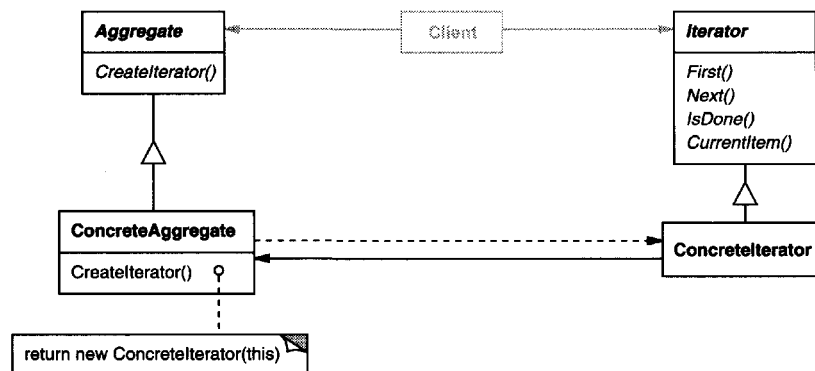
CreateIterator is an example of a factory method (see *Factory Method* (107)). We use it here to let a client ask a list object for the appropriate iterator. The *Factory Method* approach give rise to two class hierarchies, one for lists and another for iterators. The *CreateIterator* factory method "connects" the two hierarchies.

Applicability

Use the Iterator pattern

- to access an aggregate object's contents without exposing its internal representation.
- to support multiple traversals of aggregate objects.
- to provide a uniform interface for traversing different aggregate structures (that is, to support polymorphic iteration).

Structure



Participants

- **Iterator**
 - defines an interface for accessing and traversing elements.
- **ConcreteIterator**
 - implements the **Iterator** interface.
 - keeps track of the current position in the traversal of the aggregate.
- **Aggregate**
 - defines an interface for creating an **Iterator** object.
- **ConcreteAggregate**
 - implements the **Iterator** creation interface to return an instance of the proper **ConcreteIterator**.

Collaborations

- A `ConcreteIterator` keeps track of the current object in the aggregate and can compute the succeeding object in the traversal.

Consequences

The Iterator pattern has three important consequences:

1. *It supports variations in the traversal of an aggregate.* Complex aggregates may be traversed in many ways. For example, code generation and semantic checking involve traversing parse trees. Code generation may traverse the parse tree inorder or preorder. Iterators make it easy to change the traversal algorithm: Just replace the iterator instance with a different one. You can also define Iterator subclasses to support new traversals.
2. *Iterators simplify the Aggregate interface.* Iterator's traversal interface obviates the need for a similar interface in Aggregate, thereby simplifying the aggregate's interface.
3. *More than one traversal can be pending on an aggregate.* An iterator keeps track of its own traversal state. Therefore you can have more than one traversal in progress at once.

Implementation

Iterator has many implementation variants and alternatives. Some important ones follow. The trade-offs often depend on the control structures your language provides. Some languages (CLU [LG86], for example) even support this pattern directly.

1. *Who controls the iteration?* A fundamental issue is deciding which party controls the iteration, the iterator or the client that uses the iterator. When the client controls the iteration, the iterator is called an **external iterator**, and when the iterator controls it, the iterator is an **internal iterator**.² Clients that use an external iterator must advance the traversal and request the next element explicitly from the iterator. In contrast, the client hands an internal iterator an operation to perform, and the iterator applies that operation to every element in the aggregate.

External iterators are more flexible than internal iterators. It's easy to compare two collections for equality with an external iterator, for example, but it's practically impossible with internal iterators. Internal iterators are especially weak in a language like C++ that does not provide anonymous functions, closures, or continuations like Smalltalk and CLOS. But on the other hand,

²Booch refers to external and internal iterators as **active** and **passive** iterators, respectively [Boo94]. The terms "active" and "passive" describe the role of the client, not the level of activity in the iterator.

internal iterators are easier to use, because they define the iteration logic for you.

2. *Who defines the traversal algorithm?* The iterator is not the only place where the traversal algorithm can be defined. The aggregate might define the traversal algorithm and use the iterator to store just the state of the iteration. We call this kind of iterator a **cursor**, since it merely points to the current position in the aggregate. A client will invoke the Next operation on the aggregate with the cursor as an argument, and the Next operation will change the state of the cursor.³

If the iterator is responsible for the traversal algorithm, then it's easy to use different iteration algorithms on the same aggregate, and it can also be easier to reuse the same algorithm on different aggregates. On the other hand, the traversal algorithm might need to access the private variables of the aggregate. If so, putting the traversal algorithm in the iterator violates the encapsulation of the aggregate.

3. *How robust is the iterator?* It can be dangerous to modify an aggregate while you're traversing it. If elements are added or deleted from the aggregate, you might end up accessing an element twice or missing it completely. A simple solution is to copy the aggregate and traverse the copy, but that's too expensive to do in general.

A **robust iterator** ensures that insertions and removals won't interfere with traversal, and it does it without copying the aggregate. There are many ways to implement robust iterators. Most rely on registering the iterator with the aggregate. On insertion or removal, the aggregate either adjusts the internal state of iterators it has produced, or it maintains information internally to ensure proper traversal.

Kofler provides a good discussion of how robust iterators are implemented in ET++ [Kof93]. Murray discusses the implementation of robust iterators for the USL StandardComponents' List class [Mur93].

4. *Additional Iterator operations.* The minimal interface to Iterator consists of the operations First, Next, IsDone, and CurrentItem.⁴ Some additional operations might prove useful. For example, ordered aggregates can have a Previous operation that positions the iterator to the previous element. A SkipTo operation is useful for sorted or indexed collections. SkipTo positions the iterator to an object matching specific criteria.
5. *Using polymorphic iterators in C++.* Polymorphic iterators have their cost. They require the iterator object to be allocated dynamically by a factory method. Hence they should be used only when there's a need for polymorphism. Otherwise use concrete iterators, which can be allocated on the stack.

³ Cursors are a simple example of the Memento (283) pattern and share many of its implementation issues.

⁴ We can make this interface even *smaller* by merging Next, IsDone, and CurrentItem into a single operation that advances to the next object and returns it. If the traversal is finished, then this operation returns a special value (0, for instance) that marks the end of the iteration.

Polymorphic iterators have another drawback: the client is responsible for deleting them. This is error-prone, because it's easy to forget to free a heap-allocated iterator object when you're finished with it. That's especially likely when there are multiple exit points in an operation. And if an exception is triggered, the iterator object will never be freed.

The Proxy (207) pattern provides a remedy. We can use a stack-allocated proxy as a stand-in for the real iterator. The proxy deletes the iterator in its destructor. Thus when the proxy goes out of scope, the real iterator will get deallocated along with it. The proxy ensures proper cleanup, even in the face of exceptions. This is an application of the well-known C++ technique "resource allocation is initialization" [ES90]. The Sample Code gives an example.

6. *Iterators may have privileged access.* An iterator can be viewed as an extension of the aggregate that created it. The iterator and the aggregate are tightly coupled. We can express this close relationship in C++ by making the iterator a friend of its aggregate. Then you don't need to define aggregate operations whose sole purpose is to let iterators implement traversal efficiently.

However, such privileged access can make defining new traversals difficult, since it'll require changing the aggregate interface to add another friend. To avoid this problem, the Iterator class can include protected operations for accessing important but publicly unavailable members of the aggregate. Iterator subclasses (and *only* Iterator subclasses) may use these protected operations to gain privileged access to the aggregate.

7. *Iterators for composites.* External iterators can be difficult to implement over recursive aggregate structures like those in the Composite (163) pattern, because a position in the structure may span many levels of nested aggregates. Therefore an external iterator has to store a path through the Composite to keep track of the current object. Sometimes it's easier just to use an internal iterator. It can record the current position simply by calling itself recursively, thereby storing the path implicitly in the call stack.

If the nodes in a Composite have an interface for moving from a node to its siblings, parents, and children, then a cursor-based iterator may offer a better alternative. The cursor only needs to keep track of the current node; it can rely on the node interface to traverse the Composite.

Composites often need to be traversed in more than one way. Preorder, postorder, inorder, and breadth-first traversals are common. You can support each kind of traversal with a different class of iterator.

8. *Null iterators.* A **NullIterator** is a degenerate iterator that's helpful for handling boundary conditions. By definition, a NullIterator is *always* done with traversal; that is, its `IsDone` operation always evaluates to true.

NullIterator can make traversing tree-structured aggregates (like Composites) easier. At each point in the traversal, we ask the current element for an iterator for its children. Aggregate elements return a concrete iterator

as usual. But leaf elements return an instance of `NullIterator`. That lets us implement traversal over the entire structure in a uniform way.

Sample Code

We'll look at the implementation of a simple `List` class, which is part of our foundation library (Appendix C). We'll show two `Iterator` implementations, one for traversing the `List` in front-to-back order, and another for traversing back-to-front (the foundation library supports only the first one). Then we show how to use these iterators and how to avoid committing to a particular implementation. After that, we change the design to make sure iterators get deleted properly. The last example illustrates an internal iterator and compares it to its external counterpart.

1. *List and Iterator interfaces.* First let's look at the part of the `List` interface that's relevant to implementing iterators. Refer to Appendix C for the full interface.

```
template <class Item>
class List {
public:
    List(long size = DEFAULT_LIST_CAPACITY);

    long Count() const;
    Item& Get(long index) const;
    // ...
};
```

The `List` class provides a reasonably efficient way to support iteration through its public interface. It's sufficient to implement both traversals. So there's no need to give iterators privileged access to the underlying data structure; that is, the iterator classes are not friends of `List`. To enable transparent use of the different traversals we define an abstract `Iterator` class, which defines the iterator interface.

```
template <class Item>
class Iterator {
public:
    virtual void First() = 0;
    virtual void Next() = 0;
    virtual bool IsDone() const = 0;
    virtual Item CurrentItem() const = 0;
protected:
    Iterator();
};
```

2. *Iterator subclass implementations.* `ListIterator` is a subclass of `Iterator`.

```

template <class Item>
class ListIterator : public Iterator<Item> {
public:
    ListIterator(const List<Item>* aList);
    virtual void First();
    virtual void Next();
    virtual bool IsDone() const;
    virtual Item CurrentItem() const;

private:
    const List<Item>* _list;
    long _current;
};

```

The implementation of `ListIterator` is straightforward. It stores the `List` along with an index `_current` into the list:

```

template <class Item>
ListIterator<Item>::ListIterator (
    const List<Item>* aList
) : _list(aList), _current(0) {
}

```

First positions the iterator to the first element:

```

template <class Item>
void ListIterator<Item>::First () {
    _current = 0;
}

```

Next advances the current element:

```

template <class Item>
void ListIterator<Item>::Next () {
    _current++;
}

```

`IsDone` checks whether the index refers to an element within the `List`:

```

template <class Item>
bool ListIterator<Item>::IsDone () const {
    return _current >= _list->Count();
}

```

Finally, `CurrentItem` returns the item at the current index. If the iteration has already terminated, then we throw an `IteratorOutOfBounds` exception:

```

template <class Item>
Item ListIterator<Item>::CurrentItem () const {
    if (IsDone()) {
        throw IteratorOutOfBounds;
    }
    return _list->Get(_current);
}

```

The implementation of `ReverseListIterator` is identical, except its `First` operation positions `_current` to the end of the list, and `Next` decrements `_current` toward the first item.

3. *Using the iterators.* Let's assume we have a `List` of `Employee` objects, and we would like to print all the contained employees. The `Employee` class supports this with a `Print` operation. To print the list, we define a `PrintEmployees` operation that takes an iterator as an argument. It uses the iterator to traverse and print the list.

```
void PrintEmployees (Iterator<Employee*>& i) {
    for (i.First(); !i.IsDone(); i.Next()) {
        i.CurrentItem()->Print();
    }
}
```

Since we have iterators for both back-to-front and front-to-back traversals, we can reuse this operation to print the employees in both orders.

```
List<Employee*>* employees;
// ...
ListIterator<Employee*> forward(employees);
ReverseListIterator<Employee*> backward(employees);

PrintEmployees(forward);
PrintEmployees(backward);
```

4. *Avoiding commitment to a specific list implementation.* Let's consider how a `SkipList` variation of `List` would affect our iteration code. A `SkipList` subclass of `List` must provide a `SkipListIterator` that implements the `Iterator` interface. Internally, the `SkipListIterator` has to keep more than just an index to do the iteration efficiently. But since `SkipListIterator` conforms to the `Iterator` interface, the `PrintEmployees` operation can also be used when the employees are stored in a `SkipList` object.

```
SkipList<Employee*>* employees;
// ...

SkipListIterator<Employee*> iterator(employees);
PrintEmployees(iterator);
```

Although this approach works, it would be better if we didn't have to commit to a specific `List` implementation, namely `SkipList`. We can introduce an `AbstractList` class to standardize the list interface for different list implementations. `List` and `SkipList` become subclasses of `AbstractList`.

To enable polymorphic iteration, `AbstractList` defines a factory method `CreateIterator`, which subclasses override to return their corresponding iterator:

```

template <class Item>
class AbstractList {
public:
    virtual Iterator<Item>* CreateIterator() const = 0;
    // ...
};

```

An alternative would be to define a general mixin class `Traversable` that defines the interface for creating an iterator. Aggregate classes can mix in `Traversable` to support polymorphic iteration.

`List` overrides `CreateIterator` to return a `ListIterator` object:

```

template <class Item>
Iterator<Item>* List<Item>::CreateIterator () const {
    return new ListIterator<Item>(this);
}

```

Now we're in a position to write the code for printing the employees independent of a concrete representation.

```

// we know only that we have an AbstractList
AbstractList<Employee*>* employees;
// ...

Iterator<Employee*>* iterator = employees->CreateIterator();
PrintEmployees(*iterator);
delete iterator;

```

5. *Making sure iterators get deleted.* Notice that `CreateIterator` returns a newly allocated iterator object. We're responsible for deleting it. If we forget, then we've created a storage leak. To make life easier for clients, we'll provide an `IteratorPtr` that acts as a proxy for an iterator. It takes care of cleaning up the `Iterator` object when it goes out of scope.

`IteratorPtr` is always allocated on the stack.⁵ C++ automatically takes care of calling its destructor, which deletes the real iterator. `IteratorPtr` overloads both `operator->` and `operator*` in such a way that an `IteratorPtr` can be treated just like a pointer to an iterator. The members of `IteratorPtr` are all implemented inline; thus they can incur no overhead.

```

template <class Item>
class IteratorPtr {
public:
    IteratorPtr(Iterator<Item>* i): _i(i) { }
    ~IteratorPtr() { delete _i; }

```

⁵ You can ensure this at compile-time just by declaring private `new` and `delete` operators. An accompanying implementation isn't needed.

```

        Iterator<Item>* operator->() { return _i; }
        Iterator<Item>& operator*() { return *_i; }
    private:
        // disallow copy and assignment to avoid
        // multiple deletions of _i:

        IteratorPtr(const IteratorPtr&);
        IteratorPtr& operator=(const IteratorPtr&);
    private:
        Iterator<Item>* _i;
};

```

IteratorPtr lets us simplify our printing code:

```

AbstractList<Employee*>* employees;
// ...

IteratorPtr<Employee*> iterator(employees->CreateIterator());
PrintEmployees(*iterator);

```

6. *An internal ListIterator.* As a final example, let's look at a possible implementation of an internal or passive ListIterator class. Here the iterator controls the iteration, and it applies an operation to each element.

The issue in this case is how to parameterize the iterator with the operation we want to perform on each element. C++ does not support anonymous functions or closures that other languages provide for this task. There are at least two options: (1) Pass in a pointer to a function (global or static), or (2) rely on subclassing. In the first case, the iterator calls the operation passed to it at each point in the iteration. In the second case, the iterator calls an operation that a subclass overrides to enact specific behavior.

Neither option is perfect. Often you want to accumulate state during the iteration, and functions aren't well-suited to that; we would have to use static variables to remember the state. An Iterator subclass provides us with a convenient place to store the accumulated state, like in an instance variable. But creating a subclass for every different traversal is more work.

Here's a sketch of the second option, which uses subclassing. We call the internal iterator a ListTraverser.

```

template <class Item>
class ListTraverser {
public:
    ListTraverser(List<Item>* aList);
    bool Traverse();
protected:
    virtual bool ProcessItem(const Item&) = 0;
private:
    ListIterator<Item> _iterator;
};

```

ListTraverser takes a List instance as a parameter. Internally it uses an external ListIterator to do the traversal. Traverse starts the traversal

and calls `ProcessItem` for each item. The internal iterator can choose to terminate a traversal by returning `false` from `ProcessItem`. `Traverse` returns whether the traversal terminated prematurely.

```
template <class Item>
ListTraverser<Item>::ListTraverser (
    List<Item>* aList
) : _iterator(aList) { }

template <class Item>
bool ListTraverser<Item>::Traverse () {
    bool result = false;

    for (
        _iterator.First();
        !_iterator.IsDone();
        _iterator.Next()
    ) {
        result = ProcessItem(_iterator.CurrentItem());

        if (result == false) {
            break;
        }
    }
    return result;
}
```

Let's use a `ListTraverser` to print the first 10 employees from our employee list. To do it we have to subclass `ListTraverser` and override `ProcessItem`. We count the number of printed employees in a `_count` instance variable.

```
class PrintNEmployees : public ListTraverser<Employee*> {
public:
    PrintNEmployees(List<Employee*>* aList, int n) :
        ListTraverser<Employee*>(aList),
        _total(n), _count(0) { }

protected:
    bool ProcessItem(Employee* const&);
private:
    int _total;
    int _count;
};

bool PrintNEmployees::ProcessItem (Employee* const& e) {
    _count++;
    e->Print();
    return _count < _total;
}
```

Here's how `PrintNEmployees` prints the first 10 employees on the list:

```

List<Employee*>* employees;
// ...

PrintNEmployees pa(employees, 10);
pa.Traverse();

```

Note how the client doesn't specify the iteration loop. The entire iteration logic can be reused. This is the primary benefit of an internal iterator. It's a bit more work than an external iterator, though, because we have to define a new class. Contrast this with using an external iterator:

```

ListIterator<Employee*> i(employees);
int count = 0;

for (i.First(); !i.IsDone(); i.Next()) {
    count++;
    i.CurrentItem()->Print();

    if (count >= 10) {
        break;
    }
}

```

Internal iterators can encapsulate different kinds of iteration. For example, `FilteringListTraverser` encapsulates an iteration that processes only items that satisfy a test:

```

template <class Item>
class FilteringListTraverser {
public:
    FilteringListTraverser(List<Item*> aList);
    bool Traverse();
protected:
    virtual bool ProcessItem(const Item&) = 0;
    virtual bool TestItem(const Item&) = 0;
private:
    ListIterator<Item> _iterator;
};

```

This interface is the same as `ListTraverser`'s except for an added `TestItem` member function that defines the test. Subclasses override `TestItem` to specify the test.

`Traverse` decides to continue the traversal based on the outcome of the test:

```

template <class Item>
void FilteringListTraverser<Item>::Traverse () {
    bool result = false;

    for (
        _iterator.First();
        !_iterator.IsDone();
        _iterator.Next()
    ) {
        if (TestItem(_iterator.CurrentItem())) {
            result = ProcessItem(_iterator.CurrentItem());
        }
    }
}

```

```
        if (result == false) {
            break;
        }
    }
    return result;
}
```

A variant of this class could define `Traverse` to return if at least one item satisfies the test.⁶

Known Uses

Iterators are common in object-oriented systems. Most collection class libraries offer iterators in one form or another.

Here's an example from the Booch components [Boo94], a popular collection class library. It provides both a fixed size (bounded) and dynamically growing (unbounded) implementation of a queue. The queue interface is defined by an abstract `Queue` class. To support polymorphic iteration over the different queue implementations, the queue iterator is implemented in the terms of the abstract `Queue` class interface. This variation has the advantage that you don't need a factory method to ask the queue implementations for their appropriate iterator. However, it requires the interface of the abstract `Queue` class to be powerful enough to implement the iterator efficiently.

Iterators don't have to be defined as explicitly in Smalltalk. The standard collection classes (`Bag`, `Set`, `Dictionary`, `OrderedCollection`, `String`, etc.) define an internal iterator method `do:`, which takes a block (i.e., closure) as an argument. Each element in the collection is bound to the local variable in the block; then the block is executed. Smalltalk also includes a set of `Stream` classes that support an iterator-like interface. `ReadStream` is essentially an `Iterator`, and it can act as an external iterator for all the sequential collections. There are no standard external iterators for nonsequential collections such as `Set` and `Dictionary`.

Polymorphic iterators and the cleanup `Proxy` described earlier are provided by the ET++ container classes [WGM88]. The Unidraw graphical editing framework classes use cursor-based iterators [VL90].

`ObjectWindows 2.0` [Bor94] provides a class hierarchy of iterators for containers. You can iterate over different container types in the same way. The `ObjectWindow` iteration syntax relies on overloading the postincrement operator `++` to advance the iteration.

Related Patterns

Composite (163): Iterators are often applied to recursive structures such as `Composites`.

⁶The `Traverse` operation in these examples is a `Template Method` (325) with primitive operations `TestItem` and `ProcessItem`.

Factory Method (107): Polymorphic iterators rely on factory methods to instantiate the appropriate Iterator subclass.

Memento (283) is often used in conjunction with the Iterator pattern. An iterator can use a memento to capture the state of an iteration. The iterator stores the memento internally.

STRATEGY

Object Behavioral

Intent

Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

Also Known As

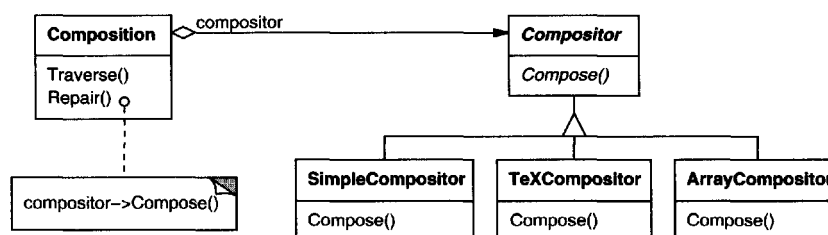
Policy

Motivation

Many algorithms exist for breaking a stream of text into lines. Hard-wiring all such algorithms into the classes that require them isn't desirable for several reasons:

- Clients that need linebreaking get more complex if they include the line-breaking code. That makes clients bigger and harder to maintain, especially if they support multiple linebreaking algorithms.
- Different algorithms will be appropriate at different times. We don't want to support multiple linebreaking algorithms if we don't use them all.
- It's difficult to add new algorithms and vary existing ones when linebreaking is an integral part of a client.

We can avoid these problems by defining classes that encapsulate different line-breaking algorithms. An algorithm that's encapsulated in this way is called a **strategy**.



Suppose a **Composition** class is responsible for maintaining and updating the linebreaks of text displayed in a text viewer. Linebreaking strategies aren't implemented by the class **Composition**. Instead, they are implemented separately by subclasses of the abstract **Compositor** class. **Compositor** subclasses implement different strategies:

- **SimpleCompositor** implements a simple strategy that determines linebreaks one at a time.
- **TeXCompositor** implements the TeX algorithm for finding linebreaks. This strategy tries to optimize linebreaks globally, that is, one paragraph at a time.
- **ArrayCompositor** implements a strategy that selects breaks so that each row has a fixed number of items. It's useful for breaking a collection of icons into rows, for example.

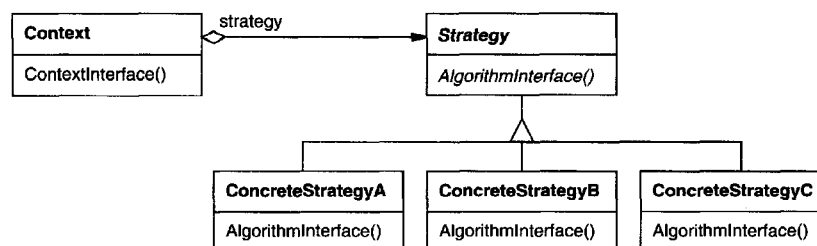
A *Composition* maintains a reference to a *Compositor* object. Whenever a *Composition* reformats its text, it forwards this responsibility to its *Compositor* object. The client of *Composition* specifies which *Compositor* should be used by installing the *Compositor* it desires into the *Composition*.

Applicability

Use the Strategy pattern when

- many related classes differ only in their behavior. Strategies provide a way to configure a class with one of many behaviors.
- you need different variants of an algorithm. For example, you might define algorithms reflecting different space/time trade-offs. Strategies can be used when these variants are implemented as a class hierarchy of algorithms [HO87].
- an algorithm uses data that clients shouldn't know about. Use the Strategy pattern to avoid exposing complex, algorithm-specific data structures.
- a class defines many behaviors, and these appear as multiple conditional statements in its operations. Instead of many conditionals, move related conditional branches into their own Strategy class.

Structure



Participants

- **Strategy** (Compositor)
 - declares an interface common to all supported algorithms. Context uses this interface to call the algorithm defined by a ConcreteStrategy.
- **ConcreteStrategy** (SimpleCompositor, TeXCompositor, ArrayCompositor)
 - implements the algorithm using the Strategy interface.
- **Context** (Composition)
 - is configured with a ConcreteStrategy object.
 - maintains a reference to a Strategy object.
 - may define an interface that lets Strategy access its data.

Collaborations

- Strategy and Context interact to implement the chosen algorithm. A context may pass all data required by the algorithm to the strategy when the algorithm is called. Alternatively, the context can pass itself as an argument to Strategy operations. That lets the strategy call back on the context as required.
- A context forwards requests from its clients to its strategy. Clients usually create and pass a ConcreteStrategy object to the context; thereafter, clients interact with the context exclusively. There is often a family of ConcreteStrategy classes for a client to choose from.

Consequences

The Strategy pattern has the following benefits and drawbacks:

1. *Families of related algorithms.* Hierarchies of Strategy classes define a family of algorithms or behaviors for contexts to reuse. Inheritance can help factor out common functionality of the algorithms.
2. *An alternative to subclassing.* Inheritance offers another way to support a variety of algorithms or behaviors. You can subclass a Context class directly to give it different behaviors. But this hard-wires the behavior into Context. It mixes the algorithm implementation with Context's, making Context harder to understand, maintain, and extend. And you can't vary the algorithm dynamically. You wind up with many related classes whose only difference is the algorithm or behavior they employ. Encapsulating the algorithm in separate Strategy classes lets you vary the algorithm independently of its context, making it easier to switch, understand, and extend.
3. *Strategies eliminate conditional statements.* The Strategy pattern offers an alternative to conditional statements for selecting desired behavior. When different behaviors are lumped into one class, it's hard to avoid using conditional

statements to select the right behavior. Encapsulating the behavior in separate Strategy classes eliminates these conditional statements.

For example, without strategies, the code for breaking text into lines could look like

```
void Composition::Repair () {
    switch (_breakingStrategy) {
        case SimpleStrategy:
            ComposeWithSimpleCompositor();
            break;
        case TeXStrategy:
            ComposeWithTeXCompositor();
            break;
        // ...
    }
    // merge results with existing composition, if necessary
}
```

The Strategy pattern eliminates this case statement by delegating the line-breaking task to a Strategy object:

```
void Composition::Repair () {
    _compositor->Compose();
    // merge results with existing composition, if necessary
}
```

Code containing many conditional statements often indicates the need to apply the Strategy pattern.

4. *A choice of implementations.* Strategies can provide different implementations of the same behavior. The client can choose among strategies with different time and space trade-offs.
5. *Clients must be aware of different Strategies.* The pattern has a potential drawback in that a client must understand how Strategies differ before it can select the appropriate one. Clients might be exposed to implementation issues. Therefore you should use the Strategy pattern only when the variation in behavior is relevant to clients.
6. *Communication overhead between Strategy and Context.* The Strategy interface is shared by all ConcreteStrategy classes whether the algorithms they implement are trivial or complex. Hence it's likely that some ConcreteStrategies won't use all the information passed to them through this interface; simple ConcreteStrategies may use none of it! That means there will be times when the context creates and initializes parameters that never get used. If this is an issue, then you'll need tighter coupling between Strategy and Context.
7. *Increased number of objects.* Strategies increase the number of objects in an application. Sometimes you can reduce this overhead by implementing strategies as stateless objects that contexts can share. Any residual state is maintained by the context, which passes it in each request to the Strategy

object. Shared strategies should not maintain state across invocations. The Flyweight (195) pattern describes this approach in more detail.

Implementation

Consider the following implementation issues:

1. *Defining the Strategy and Context interfaces.* The Strategy and Context interfaces must give a ConcreteStrategy efficient access to any data it needs from a context, and vice versa.

One approach is to have Context pass data in parameters to Strategy operations—in other words, take the data to the strategy. This keeps Strategy and Context decoupled. On the other hand, Context might pass data the Strategy doesn't need.

Another technique has a context pass *itself* as an argument, and the strategy requests data from the context explicitly. Alternatively, the strategy can store a reference to its context, eliminating the need to pass anything at all. Either way, the strategy can request exactly what it needs. But now Context must define a more elaborate interface to its data, which couples Strategy and Context more closely.

The needs of the particular algorithm and its data requirements will determine the best technique.

2. *Strategies as template parameters.* In C++ templates can be used to configure a class with a strategy. This technique is only applicable if (1) the Strategy can be selected at compile-time, and (2) it does not have to be changed at run-time. In this case, the class to be configured (e.g., Context) is defined as a template class that has a Strategy class as a parameter:

```
template <class AStrategy>
class Context {
    void Operation() { theStrategy.DoAlgorithm(); }
    // ...
private:
    AStrategy theStrategy;
};
```

The class is then configured with a Strategy class when it's instantiated:

```
class MyStrategy {
public:
    void DoAlgorithm();
};

Context<MyStrategy> aContext;
```

With templates, there's no need to define an abstract class that defines the interface to the Strategy. Using Strategy as a template parameter also lets you bind a Strategy to its Context statically, which can increase efficiency.

3. *Making Strategy objects optional.* The Context class may be simplified if it's meaningful *not* to have a Strategy object. Context checks to see if it has a Strategy object before accessing it. If there is one, then Context uses it normally. If there isn't a strategy, then Context carries out default behavior. The benefit of this approach is that clients don't have to deal with Strategy objects at all *unless* they don't like the default behavior.

Sample Code

We'll give the high-level code for the Motivation example, which is based on the implementation of Composition and Compositor classes in InterViews [LCI⁺92].

The Composition class maintains a collection of Component instances, which represent text and graphical elements in a document. A composition arranges component objects into lines using an instance of a Compositor subclass, which encapsulates a linebreaking strategy. Each component has an associated natural size, stretchability, and shrinkability. The stretchability defines how much the component can grow beyond its natural size; shrinkability is how much it can shrink. The composition passes these values to a compositor, which uses them to determine the best location for linebreaks.

```
class Composition {
public:
    Composition(Compositor*);
    void Repair();
private:
    Compositor* _compositor;
    Component* _components;    // the list of components
    int _componentCount;       // the number of components
    int _lineWidth;            // the Composition's line width
    int* _lineBreaks;          // the position of linebreaks
                                // in components
    int _lineCount;            // the number of lines
};
```

When a new layout is required, the composition asks its compositor to determine where to place linebreaks. The composition passes the compositor three arrays that define natural sizes, stretchabilities, and shrinkabilities of the components. It also passes the number of components, how wide the line is, and an array that the compositor fills with the position of each linebreak. The compositor returns the number of calculated breaks.

The Compositor interface lets the composition pass the compositor all the information it needs. This is an example of "taking the data to the strategy":

```

class Compositor {
public:
    virtual int Compose(
        Coord natural[], Coord stretch[], Coord shrink[],
        int componentCount, int lineWidth, int breaks[]
    ) = 0;
protected:
    Compositor();
};

```

Note that `Compositor` is an abstract class. Concrete subclasses define specific linebreaking strategies.

The composition calls its compositor in its `Repair` operation. `Repair` first initializes arrays with the natural size, stretchability, and shrinkability of each component (the details of which we omit for brevity). Then it calls on the compositor to obtain the linebreaks and finally lays out the components according to the breaks (also omitted):

```

void Composition::Repair () {
    Coord* natural;
    Coord* stretchability;
    Coord* shrinkability;
    int componentCount;
    int* breaks;

    // prepare the arrays with the desired component sizes
    // ...

    // determine where the breaks are:
    int breakCount;
    breakCount = _compositor->Compose(
        natural, stretchability, shrinkability,
        componentCount, _lineWidth, breaks
    );

    // lay out components according to breaks
    // ...
}

```

Now let's look at the `Compositor` subclasses. `SimpleCompositor` examines components a line at a time to determine where breaks should go:

```

class SimpleCompositor : public Compositor {
public:
    SimpleCompositor();

    virtual int Compose(
        Coord natural[], Coord stretch[], Coord shrink[],
        int componentCount, int lineWidth, int breaks[]
    );
    // ...
};

```

TeXCompositor uses a more global strategy. It examines a *paragraph* at a time, taking into account the components' size and stretchability. It also tries to give an even "color" to the paragraph by minimizing the whitespace between components.

```
class TeXCompositor : public Compositor {
public:
    TeXCompositor();

    virtual int Compose(
        Coord natural[], Coord stretch[], Coord shrink[],
        int componentCount, int lineWidth, int breaks[]
    );
    // ...
};
```

ArrayCompositor breaks the components into lines at regular intervals.

```
class ArrayCompositor : public Compositor {
public:
    ArrayCompositor(int interval);

    virtual int Compose(
        Coord natural[], Coord stretch[], Coord shrink[],
        int componentCount, int lineWidth, int breaks[]
    );
    // ...
};
```

These classes don't use all the information passed in Compose. SimpleCompositor ignores the stretchability of the components, taking only their natural widths into account. TeXCompositor uses all the information passed to it, whereas ArrayCompositor ignores everything.

To instantiate Composition, you pass it the compositor you want to use:

```
Composition* quick = new Composition(new SimpleCompositor);
Composition* slick = new Composition(new TeXCompositor);
Composition* iconic = new Composition(new ArrayCompositor(100));
```

Compositor's interface is carefully designed to support all layout algorithms that subclasses might implement. You don't want to have to change this interface with every new subclass, because that will require changing existing subclasses. In general, the Strategy and Context interfaces determine how well the pattern achieves its intent.

Known Uses

Both ET++ [WGM88] and InterViews use strategies to encapsulate different line-breaking algorithms as we've described.

In the RTL System for compiler code optimization [JML92], strategies define different register allocation schemes (RegisterAllocator) and instruction set scheduling policies (RISCscheduler, CISCscheduler). This provides flexibility in targeting the optimizer for different machine architectures.

The ET++SwapsManager calculation engine framework computes prices for different financial instruments [EG92]. Its key abstractions are Instrument and YieldCurve. Different instruments are implemented as subclasses of Instrument. YieldCurve calculates discount factors, which determine the present value of future cash flows. Both of these classes delegate some behavior to Strategy objects. The framework provides a family of ConcreteStrategy classes for generating cash flows, valuing swaps, and calculating discount factors. You can create new calculation engines by configuring Instrument and YieldCurve with the different ConcreteStrategy objects. This approach supports mixing and matching existing Strategy implementations as well as defining new ones.

The Booch components [BV90] use strategies as template arguments. The Booch collection classes support three different kinds of memory allocation strategies: managed (allocation out of a pool), controlled (allocations/deallocations are protected by locks), and unmanaged (the normal memory allocator). These strategies are passed as template arguments to a collection class when it's instantiated. For example, an UnboundedCollection that uses the unmanaged strategy is instantiated as `UnboundedCollection<MyItemType*, Unmanaged>`.

RApp is a system for integrated circuit layout [GA89, AG90]. RApp must lay out and route wires that connect subsystems on the circuit. Routing algorithms in RApp are defined as subclasses of an abstract Router class. Router is a Strategy class.

Borland's ObjectWindows [Bor94] uses strategies in dialogs boxes to ensure that the user enters valid data. For example, numbers might have to be in a certain range, and a numeric entry field should accept only digits. Validating that a string is correct can require a table look-up.

ObjectWindows uses Validator objects to encapsulate validation strategies. Validators are examples of Strategy objects. Data entry fields delegate the validation strategy to an optional Validator object. The client attaches a validator to a field if validation is required (an example of an optional strategy). When the dialog is closed, the entry fields ask their validators to validate the data. The class library provides validators for common cases, such as a RangeValidator for numbers. New client-specific validation strategies can be defined easily by subclassing the Validator class.

Related Patterns

Flyweight (195): Strategy objects often make good flyweights.

VISITOR

Object Behavioral

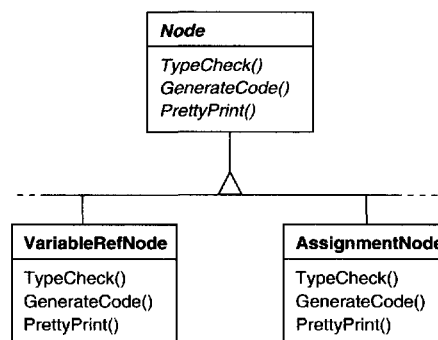
Intent

Represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.

Motivation

Consider a compiler that represents programs as abstract syntax trees. It will need to perform operations on abstract syntax trees for “static semantic” analyses like checking that all variables are defined. It will also need to generate code. So it might define operations for type-checking, code optimization, flow analysis, checking for variables being assigned values before they’re used, and so on. Moreover, we could use the abstract syntax trees for pretty-printing, program restructuring, code instrumentation, and computing various metrics of a program.

Most of these operations will need to treat nodes that represent assignment statements differently from nodes that represent variables or arithmetic expressions. Hence there will be one class for assignment statements, another for variable accesses, another for arithmetic expressions, and so on. The set of node classes depends on the language being compiled, of course, but it doesn’t change much for a given language.



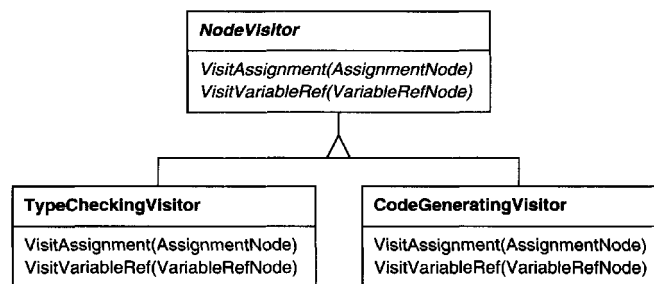
This diagram shows part of the Node class hierarchy. The problem here is that distributing all these operations across the various node classes leads to a system that’s hard to understand, maintain, and change. It will be confusing to have type-checking code mixed with pretty-printing code or flow analysis code. Moreover, adding a new operation usually requires recompiling all of these classes. It would

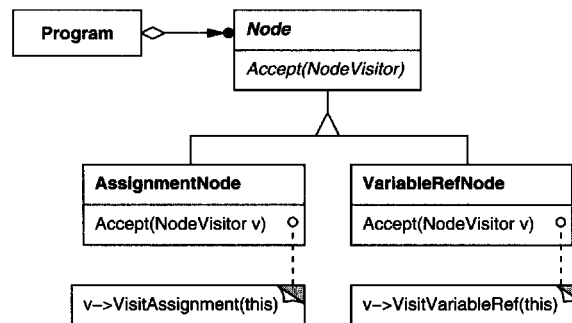
be better if each new operation could be added separately, and the node classes were independent of the operations that apply to them.

We can have both by packaging related operations from each class in a separate object, called a **visitor**, and passing it to elements of the abstract syntax tree as it's traversed. When an element "accepts" the visitor, it sends a request to the visitor that encodes the element's class. It also includes the element as an argument. The visitor will then execute the operation for that element—the operation that used to be in the class of the element.

For example, a compiler that didn't use visitors might type-check a procedure by calling the `TypeCheck` operation on its abstract syntax tree. Each of the nodes would implement `TypeCheck` by calling `TypeCheck` on its components (see the preceding class diagram). If the compiler type-checked a procedure using visitors, then it would create a `TypeCheckingVisitor` object and call the `Accept` operation on the abstract syntax tree with that object as an argument. Each of the nodes would implement `Accept` by calling back on the visitor: an assignment node calls `VisitAssignment` operation on the visitor, while a variable reference calls `VisitVariableReference`. What used to be the `TypeCheck` operation in class `AssignmentNode` is now the `VisitAssignment` operation on `TypeCheckingVisitor`.

To make visitors work for more than just type-checking, we need an abstract parent class `NodeVisitor` for all visitors of an abstract syntax tree. `NodeVisitor` must declare an operation for each node class. An application that needs to compute program metrics will define new subclasses of `NodeVisitor` and will no longer need to add application-specific code to the node classes. The Visitor pattern encapsulates the operations for each compilation phase in a Visitor associated with that phase.





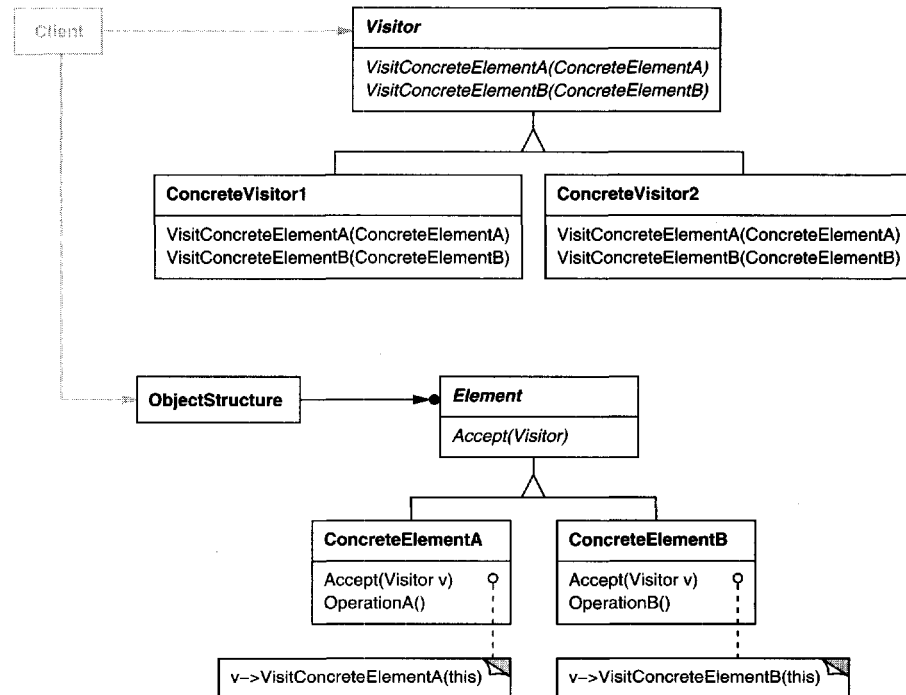
With the Visitor pattern, you define two class hierarchies: one for the elements being operated on (the Node hierarchy) and one for the visitors that define operations on the elements (the NodeVisitor hierarchy). You create a new operation by adding a new subclass to the visitor class hierarchy. As long as the grammar that the compiler accepts doesn't change (that is, we don't have to add new Node subclasses), we can add new functionality simply by defining new NodeVisitor subclasses.

Applicability

Use the Visitor pattern when

- an object structure contains many classes of objects with differing interfaces, and you want to perform operations on these objects that depend on their concrete classes.
- many distinct and unrelated operations need to be performed on objects in an object structure, and you want to avoid "polluting" their classes with these operations. Visitor lets you keep related operations together by defining them in one class. When the object structure is shared by many applications, use Visitor to put operations in just those applications that need them.
- the classes defining the object structure rarely change, but you often want to define new operations over the structure. Changing the object structure classes requires redefining the interface to all visitors, which is potentially costly. If the object structure classes change often, then it's probably better to define the operations in those classes.

Structure



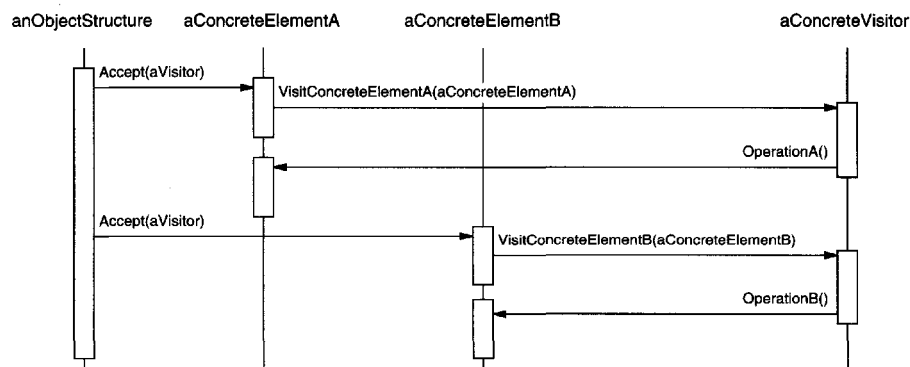
Participants

- **Visitor** (NodeVisitor)
 - declares a Visit operation for each class of ConcreteElement in the object structure. The operation's name and signature identifies the class that sends the Visit request to the visitor. That lets the visitor determine the concrete class of the element being visited. Then the visitor can access the element directly through its particular interface.
- **ConcreteVisitor** (TypeCheckingVisitor)
 - implements each operation declared by Visitor. Each operation implements a fragment of the algorithm defined for the corresponding class of object in the structure. ConcreteVisitor provides the context for the algorithm and stores its local state. This state often accumulates results during the traversal of the structure.
- **Element** (Node)
 - defines an Accept operation that takes a visitor as an argument.

- **ConcreteElement** (AssignmentNode, VariableRefNode)
 - implements an Accept operation that takes a visitor as an argument.
- **ObjectStructure** (Program)
 - can enumerate its elements.
 - may provide a high-level interface to allow the visitor to visit its elements.
 - may either be a composite (see Composite (163)) or a collection such as a list or a set.

Collaborations

- A client that uses the Visitor pattern must create a ConcreteVisitor object and then traverse the object structure, visiting each element with the visitor.
 - When an element is visited, it calls the Visitor operation that corresponds to its class. The element supplies itself as an argument to this operation to let the visitor access its state, if necessary.
- The following interaction diagram illustrates the collaborations between an object structure, a visitor, and two elements:



Consequences

Some of the benefits and liabilities of the Visitor pattern are as follows:

1. *Visitor makes adding new operations easy.* Visitors make it easy to add operations that depend on the components of complex objects. You can define a new operation over an object structure simply by adding a new visitor. In contrast, if you spread functionality over many classes, then you must change each class to define a new operation.
2. *A visitor gathers related operations and separates unrelated ones.* Related behavior isn't spread over the classes defining the object structure; it's localized in a visitor. Unrelated sets of behavior are partitioned in their own visitor.

subclasses. That simplifies both the classes defining the elements and the algorithms defined in the visitors. Any algorithm-specific data structures can be hidden in the visitor.

3. *Adding new ConcreteElement classes is hard.* The Visitor pattern makes it hard to add new subclasses of Element. Each new ConcreteElement gives rise to a new abstract operation on Visitor and a corresponding implementation in every ConcreteVisitor class. Sometimes a default implementation can be provided in Visitor that can be inherited by most of the ConcreteVisitors, but this is the exception rather than the rule.

So the key consideration in applying the Visitor pattern is whether you are mostly likely to change the algorithm applied over an object structure or the classes of objects that make up the structure. The Visitor class hierarchy can be difficult to maintain when new ConcreteElement classes are added frequently. In such cases, it's probably easier just to define operations on the classes that make up the structure. If the Element class hierarchy is stable, but you are continually adding operations or changing algorithms, then the Visitor pattern will help you manage the changes.

4. *Visiting across class hierarchies.* An iterator (see Iterator (257)) can visit the objects in a structure as it traverses them by calling their operations. But an iterator can't work across object structures with different types of elements. For example, the Iterator interface defined on page 263 can access only objects of type Item:

```
template <class Item>
class Iterator {
    // ...
    Item CurrentItem() const;
};
```

This implies that all elements the iterator can visit have a common parent class Item.

Visitor does not have this restriction. It can visit objects that don't have a common parent class. You can add any type of object to a Visitor interface. For example, in

```
class Visitor {
public:
    // ...
    void VisitMyType(MyType*);
    void VisitYourType(YourType*);
};
```

MyType and YourType do not have to be related through inheritance at all.

5. *Accumulating state.* Visitors can accumulate state as they visit each element in the object structure. Without a visitor, this state would be passed as extra arguments to the operations that perform the traversal, or they might appear as global variables.

6. *Breaking encapsulation.* Visitor's approach assumes that the ConcreteElement interface is powerful enough to let visitors do their job. As a result, the pattern often forces you to provide public operations that access an element's internal state, which may compromise its encapsulation.

Implementation

Each object structure will have an associated Visitor class. This abstract visitor class declares a VisitConcreteElement operation for each class of ConcreteElement defining the object structure. Each Visit operation on the Visitor declares its argument to be a particular ConcreteElement, allowing the Visitor to access the interface of the ConcreteElement directly. ConcreteVisitor classes override each Visit operation to implement visitor-specific behavior for the corresponding ConcreteElement class.

The Visitor class would be declared like this in C++:

```
class Visitor {
public:
    virtual void VisitElementA(ElementA*);
    virtual void VisitElementB(ElementB*);

    // and so on for other concrete elements
protected:
    Visitor();
};
```

Each class of ConcreteElement implements an Accept operation that calls the matching Visit... operation on the visitor for that ConcreteElement. Thus the operation that ends up getting called depends on both the class of the element and the class of the visitor.¹⁰

The concrete elements are declared as

```
class Element {
public:
    virtual ~Element();
    virtual void Accept(Visitor&) = 0;
protected:
    Element();
};
```

¹⁰ We could use function overloading to give these operations the same simple name, like Visit, since the operations are already differentiated by the parameter they're passed. There are pros and cons to such overloading. On the one hand, it reinforces the fact that each operation involves the same analysis, albeit on a different argument. On the other hand, that might make what's going on at the call site less obvious to someone reading the code. It really boils down to whether you believe function overloading is good or not.

```

class ElementA : public Element {
public:
    ElementA();
    virtual void Accept(Visitor& v) { v.VisitElementA(this); }
};

class ElementB : public Element {
public:
    ElementB();
    virtual void Accept(Visitor& v) { v.VisitElementB(this); }
};

```

A CompositeElement class might implement Accept like this:

```

class CompositeElement : public Element {
public:
    virtual void Accept(Visitor&);
private:
    List<Element*>* _children;
};

void CompositeElement::Accept (Visitor& v) {
    ListIterator<Element*> i(_children);

    for (i.First(); !i.IsDone(); i.Next()) {
        i.CurrentItem()->Accept(v);
    }
    v.VisitCompositeElement(this);
}

```

Here are two other implementation issues that arise when you apply the Visitor pattern:

1. *Double dispatch.* Effectively, the Visitor pattern lets you add operations to classes without changing them. Visitor achieves this by using a technique called **double-dispatch**. It's a well-known technique. In fact, some programming languages support it directly (CLOS, for example). Languages like C++ and Smalltalk support **single-dispatch**.

In single-dispatch languages, two criteria determine which operation will fulfill a request: the name of the request and the type of receiver. For example, the operation that a GenerateCode request will call depends on the type of node object you ask. In C++, calling GenerateCode on an instance of VariableRefNode will call VariableRefNode::GenerateCode (which generates code for a variable reference). Calling GenerateCode on an AssignmentNode will call AssignmentNode::GenerateCode (which will generate code for an assignment). The operation that gets executed depends both on the kind of request and the type of the receiver.

"Double-dispatch" simply means the operation that gets executed depends on the kind of request and the types of *two* receivers. Accept is a double-dispatch operation. Its meaning depends on two types: the Visitor's and the

Element's. Double-dispatching lets visitors request different operations on each class of element.¹¹

This is the key to the Visitor pattern: The operation that gets executed depends on both the type of Visitor and the type of Element it visits. Instead of binding operations statically into the Element interface, you can consolidate the operations in a Visitor and use `Accept` to do the binding at run-time. Extending the Element interface amounts to defining one new Visitor subclass rather than many new Element subclasses.

2. *Who is responsible for traversing the object structure?* A visitor must visit each element of the object structure. The question is, how does it get there? We can put responsibility for traversal in any of three places: in the object structure, in the visitor, or in a separate iterator object (see Iterator (257)).

Often the object structure is responsible for iteration. A collection will simply iterate over its elements, calling the `Accept` operation on each. A composite will commonly traverse itself by having each `Accept` operation traverse the element's children and call `Accept` on each of them recursively.

Another solution is to use an iterator to visit the elements. In C++, you could use either an internal or external iterator, depending on what is available and what is most efficient. In Smalltalk, you usually use an internal iterator using `do:` and a block. Since internal iterators are implemented by the object structure, using an internal iterator is a lot like making the object structure responsible for iteration. The main difference is that an internal iterator will not cause double-dispatching—it will call an operation on the *visitor* with an *element* as an argument as opposed to calling an operation on the *element* with the *visitor* as an argument. But it's easy to use the Visitor pattern with an internal iterator if the operation on the visitor simply calls the operation on the element without recursing.

You could even put the traversal algorithm in the visitor, although you'll end up duplicating the traversal code in each `ConcreteVisitor` for each aggregate `ConcreteElement`. The main reason to put the traversal strategy in the visitor is to implement a particularly complex traversal, one that depends on the results of the operations on the object structure. We'll give an example of such a case in the Sample Code.

Sample Code

Because visitors are usually associated with composites, we'll use the `Equipment` classes defined in the Sample Code of Composite (163) to illustrate the Visitor pattern. We will use Visitor to define operations for computing the inventory of materials and the total cost for a piece of equipment. The `Equipment` classes are

¹¹If we can have *double-dispatch*, then why not *triple* or *quadruple*, or any other number? Actually, double-dispatch is just a special case of **multiple dispatch**, in which the operation is chosen based on any number of types. (CLOS actually supports multiple dispatch.) Languages that support double- or multiple dispatch lessen the need for the Visitor pattern.

so simple that using Visitor isn't really necessary, but they make it easy to see what's involved in implementing the pattern.

Here again is the `Equipment` class from Composite (163). We've augmented it with an `Accept` operation to let it work with a visitor.

```
class Equipment {
public:
    virtual ~Equipment();

    const char* Name() { return _name; }

    virtual Watt Power();
    virtual Currency NetPrice();
    virtual Currency DiscountPrice();

    virtual void Accept(EquipmentVisitor&);
protected:
    Equipment(const char*);
private:
    const char* _name;
};
```

The `Equipment` operations return the attributes of a piece of equipment, such as its power consumption and cost. Subclasses redefine these operations appropriately for specific types of equipment (e.g., a chassis, drives, and planar boards).

The abstract class for all visitors of equipment has a virtual function for each subclass of equipment, as shown next. All of the virtual functions do nothing by default.

```
class EquipmentVisitor {
public:
    virtual ~EquipmentVisitor();

    virtual void VisitFloppyDisk(FloppyDisk*);
    virtual void VisitCard(Card*);
    virtual void VisitChassis(Chassis*);
    virtual void VisitBus(Bus*);

    // and so on for other concrete subclasses of Equipment
protected:
    EquipmentVisitor();
};
```

`Equipment` subclasses define `Accept` in basically the same way: It calls the `EquipmentVisitor` operation that corresponds to the class that received the `Accept` request, like this:

```
void FloppyDisk::Accept (EquipmentVisitor& visitor) {
    visitor.VisitFloppyDisk(this);
}
```

Equipment that contains other equipment (in particular, subclasses of CompositeEquipment in the Composite pattern) implements Accept by iterating over its children and calling Accept on each of them. Then it calls the Visit operation as usual. For example, Chassis::Accept could traverse all the parts in the chassis as follows:

```
void Chassis::Accept (EquipmentVisitor& visitor) {
    for (
        ListIterator<Equipment*> i(_parts);
        !i.IsDone();
        i.Next()
    ) {
        i.CurrentItem()->Accept(visitor);
    }
    visitor.VisitChassis(this);
}
```

Subclasses of EquipmentVisitor define particular algorithms over the equipment structure. The PricingVisitor computes the cost of the equipment structure. It computes the net price of all simple equipment (e.g., floppies) and the discount price of all composite equipment (e.g., chassis and buses).

```
class PricingVisitor : public EquipmentVisitor {
public:
    PricingVisitor();

    Currency& GetTotalPrice();

    virtual void VisitFloppyDisk(FloppyDisk*);
    virtual void VisitCard(Card*);
    virtual void VisitChassis(Chassis*);
    virtual void VisitBus(Bus*);
    // ...
private:
    Currency _total;
};

void PricingVisitor::VisitFloppyDisk (FloppyDisk* e) {
    _total += e->NetPrice();
}

void PricingVisitor::VisitChassis (Chassis* e) {
    _total += e->DiscountPrice();
}
```

PricingVisitor will compute the total cost of all nodes in the equipment structure. Note that PricingVisitor chooses the appropriate pricing policy for a class of equipment by dispatching to the corresponding member function. What's more, we can change the pricing policy of an equipment structure just by changing the PricingVisitor class.

We can define a visitor for computing inventory like this:

```
class InventoryVisitor : public EquipmentVisitor {
public:
    InventoryVisitor();

    Inventory& GetInventory();

    virtual void VisitFloppyDisk(FloppyDisk*);
    virtual void VisitCard(Card*);
    virtual void VisitChassis(Chassis*);
    virtual void VisitBus(Bus*);
    // ...

private:
    Inventory _inventory;
};
```

The `InventoryVisitor` accumulates the totals for each type of equipment in the object structure. `InventoryVisitor` uses an `Inventory` class that defines an interface for adding equipment (which we won't bother defining here).

```
void InventoryVisitor::VisitFloppyDisk (FloppyDisk* e) {
    _inventory.Accumulate(e);
}

void InventoryVisitor::VisitChassis (Chassis* e) {
    _inventory.Accumulate(e);
}
```

Here's how we can use an `InventoryVisitor` on an equipment structure:

```
Equipment* component;
InventoryVisitor visitor;

component->Accept(visitor);
cout << "Inventory "
     << component->Name()
     << visitor.GetInventory();
```

Now we'll show how to implement the Smalltalk example from the Interpreter pattern (see page 248) with the Visitor pattern. Like the previous example, this one is so small that Visitor probably won't buy us much, but it provides a good illustration of how to use the pattern. Further, it illustrates a situation in which iteration is the visitor's responsibility.

The object structure (regular expressions) is made of four classes, and all of them have an `accept` : method that takes the visitor as an argument. In class `SequenceExpression`, the `accept` : method is

```

accept: aVisitor
  ^ aVisitor visitSequence: self

```

In class `RepeatExpression`, the `accept:` method sends the `visitRepeat:` message. In class `AlternationExpression`, it sends the `visitAlternation:` message. In class `LiteralExpression`, it sends the `visitLiteral:` message.

The four classes also must have accessing functions that the visitor can use. For `SequenceExpression` these are `expression1` and `expression2`; for `AlternationExpression` these are `alternative1` and `alternative2`; for `RepeatExpression` it is `repetition`; and for `LiteralExpression` these are components.

The ConcreteVisitor class is `REMatchingVisitor`. It is responsible for the traversal because its traversal algorithm is irregular. The biggest irregularity is that a `RepeatExpression` will repeatedly traverse its component. The class `REMatchingVisitor` has an instance variable `inputState`. Its methods are essentially the same as the `match:` methods of the expression classes in the Interpreter pattern except they replace the argument named `inputState` with the expression node being matched. However, they still return the set of streams that the expression would match to identify the current state.

```

visitSequence: sequenceExp
  inputState := sequenceExp expression1 accept: self.
  ^ sequenceExp expression2 accept: self.

visitRepeat: repeatExp
  | finalState |
  finalState := inputState copy.
  [inputState isEmpty]
  whileFalse:
    [inputState := repeatExp repetition accept: self.
     finalState addAll: inputState].
  ^ finalState

visitAlternation: alternateExp
  | finalState originalState |
  originalState := inputState.
  finalState := alternateExp alternative1 accept: self.
  inputState := originalState.
  finalState addAll: (alternateExp alternative2 accept: self).
  ^ finalState

```

```

visitLiteral: literalExp
| finalState tStream |
finalState := Set new.
inputState
do:
[:stream | tStream := stream copy.
(tStream nextAvailable:
literalExp components size
) = literalExp components
ifTrue: [finalState add: tStream]
].
^ finalState

```

Known Uses

The Smalltalk-80 compiler has a Visitor class called `ProgramNodeEnumerator`. It's used primarily for algorithms that analyze source code. It isn't used for code generation or pretty-printing, although it could be.

IRIS Inventor [Str93] is a toolkit for developing 3-D graphics applications. Inventor represents a three-dimensional scene as a hierarchy of nodes, each representing either a geometric object or an attribute of one. Operations like rendering a scene or mapping an input event require traversing this hierarchy in different ways. Inventor does this using visitors called "actions." There are different visitors for rendering, event handling, searching, filing, and determining bounding boxes.

To make adding new nodes easier, Inventor implements a double-dispatch scheme for C++. The scheme relies on run-time type information and a two-dimensional table in which rows represent visitors and columns represent node classes. The cells store a pointer to the function bound to the visitor and node class.

Mark Linton coined the term "Visitor" in the X Consortium's Fresco Application Toolkit specification [LP93].

Related Patterns

Composite (163): Visitors can be used to apply an operation over an object structure defined by the Composite pattern.

Interpreter (243): Visitor may be applied to do the interpretation.